

Block 2: Stacks, Queues and Trees

Short explanations, worked examples and self-check questions for every unit, with every idea also explained in Malayalam.

ഓരോ ആശയവും ലളിതമായ ഇംഗ്ലീഷിലും മലയാളത്തിലും, ഉദാഹരണങ്ങളോടെ.

UNIT 4

Stacks

ഒരു അറ്റത്ത് (top) മാത്രം insert-ഉം delete-ഉം നടക്കുന്ന LIFO data structure ആയ stack ആണ് ഈ unit-ൽ. Array, linked list എന്നിവ ഉപയോഗിച്ച് stack implement ചെയ്യുന്നതും, ഒരു array-ൽ പല stack-കൾ വെക്കുന്നതും, expression conversion, evaluation, recursion പോലുള്ള applications-ഉം പഠിക്കുന്നു.

Stack: LIFO, push and pop

Stack as an Abstract Data Type

Push and pop algorithms; overflow and underflow

Implementation of stack using arrays

Implementation of stack using linked lists

Array stack vs linked stack

Two stacks in one array

Multiple (n) stacks in one array

Applications; infix, prefix and postfix notation

Converting infix to postfix

Converting infix to prefix

Evaluating a postfix expression

Stacks and recursive function calls

Stack: LIFO, push and pop

Stack: a linear data structure in which items are inserted and removed only at one end, called the top; so the last item inserted is the first removed (LIFO).

Push: the operation that inserts an element at the top of the stack. **Pop:** the operation that deletes the element at the top of the stack.

Stack: top എന്ന ഒരേയൊരു അറ്റത്ത് മാത്രം item-കൾ insert ചെയ്യുകയും remove ചെയ്യുകയും ചെയ്യുന്ന linear data structure; അതിനാൽ അവസാനം insert ചെയ്ത item ആണ് ആദ്യം remove ആകുന്നത് (LIFO).

Push: stack-ന്റെ top-ൽ ഒരു element insert ചെയ്യുന്ന operation. **Pop:** stack-ന്റെ top-ലെ element delete ചെയ്യുന്ന operation.

A stack of dishes is the unit's picture: you can add or remove a dish only at the top. The item added last is removed first, so a stack is also called a **LIFO** (Last In First Out) or **FILO** (First In Last Out) list, and also a **pile** or **push-down list**. A variable **top** keeps track of where the next insertion or removal happens. Popping the only item of a stack gives the **empty stack**; pop cannot be applied to an empty stack, but push can be applied to any stack (in the abstract, a stack has no upper limit).

മലയാളത്തിൽ

കല്യാണവീട്ടിൽ അടുക്കിവെച്ച പ്ലേറ്റുകൾ ഓർത്തുനോക്കൂ. പുതിയ പ്ലേറ്റ് മുകളിലേക്കേ വെക്കൂ, എടുക്കുന്നതും മുകളിൽ നിന്നേ എടുക്കൂ. അതുകൊണ്ട് അവസാനം വെച്ച പ്ലേറ്റ് ആദ്യം പുറത്തുവരും; ഇതാണ് LIFO. മുകളിലേക്ക് വെക്കുന്നത് push, മുകളിൽ നിന്ന് എടുക്കുന്നത് pop. ഒന്നുമില്ലാത്ത stack-ൽ നിന്ന് pop ചെയ്യാൻ പറ്റില്ല, പക്ഷേ push എപ്പോഴും ചെയ്യാം.

LIFO = Last In First Out = FILO = First In Last Out

Push: insert at top, then top moves up by 1

Pop: remove from top, then top moves down by 1

EXAMPLE 1 (THE UNIT'S EXAMPLE 4.1)

Start with an empty stack (top = 0) and the list A, B, C. Push A, B, C one by one, then pop three times.

Operation	Stack (bottom → top)	top	List left
start	empty	0	A, B, C
push A	A	1	B, C
push B	A B	2	C
push C	A B C	3	none
pop → C	A B	2	C
pop → B	A	1	B, C
pop → A	empty	0	A, B, C

The items come out in the order C, B, A: the reverse of the order they went in.

EXAMPLE 2

On an empty stack perform: push 5, push 8, pop, push 3, push 9, pop, pop. What is popped each time, and what is left?

push 5 → [5]; push 8 → [5, 8]; pop gives 8 → [5]; push 3 → [5, 3]; push 9 → [5, 3, 9]; pop gives 9 → [5, 3]; pop gives 3 → [5]. Popped: 8, 9, 3. Left: 5 (at the top, top = 1).

Exam tip: A stack is LIFO, not FIFO (FIFO is a queue). Reversing a sequence is the classic use of LIFO: push every item, then pop every item.

Stack as an Abstract Data Type

Stack ADT: a stack described only by its operations (create, push, pop, read the top item, test for empty) and their LIFO behaviour, independent of how it is stored.

Stack ADT: എങ്ങനെ store ചെയ്യുന്നു എന്ന് നോക്കാതെ, create, push, pop, top item വായിക്കൽ, empty ആണോ എന്ന് നോക്കൽ എന്നീ operations-ഉം അവയുടെ LIFO സ്വഭാവവും കൊണ്ട് മാത്രം വിവരിക്കുന്ന stack.

The unit compares the stack ADT to piles on a desk: one pile of bills to pay, one of magazines, one of notes. With such a pile we can start a new pile, put something on top, take the top item off, look at the top item, and check

whether the pile is empty. The names push and pop come from a spring-loaded rack of cafeteria trays: trays are pushed down onto the spring, and when a tray is taken the rest pop up.

മലയാളത്തിൽ

ADT എന്നാൽ "എന്തൊക്കെ ചെയ്യാം" എന്നതു മാത്രം പറയുന്ന ഒരു വിവരണം; "എങ്ങനെ ചെയ്യുന്നു" എന്നത് പിന്നീട് array കൊണ്ടോ linked list കൊണ്ടോ തീരുമാനിക്കാം. ഹോട്ടലിലെ spring ഉള്ള tray rack-ൽ tray വെക്കുമ്പോൾ താഴേക്ക് അമരുന്നു (push), ഒന്ന് എടുക്കുമ്പോൾ ബാക്കി മുകളിലേക്ക് പൊങ്ങുന്നു (pop). അങ്ങനെയാണ് ഈ പേരുകൾ വന്നത്.

create() → a new empty stack
push(S, x) → put x on top of S
pop(S) → remove and return the top item of S (error if S is empty)
peek(S) / top(S) → return the top item without removing it
isEmpty(S) → true if S has no items

EXAMPLE

S is empty. Give the result of: push(S, 4); push(S, 7); peek(S); pop(S); isEmpty(S); pop(S); isEmpty(S).
peek gives 7 (S stays [4, 7]); pop gives 7 (S = [4]); isEmpty is false; pop gives 4 (S = []); isEmpty is true.

Exam tip: peek (read the top) and pop are different: peek leaves the item on the stack, pop removes it.

Push and pop algorithms; overflow and underflow

Stack overflow: the error condition when a push is attempted on a stack that is already full.

Stack underflow: the error condition when a pop is attempted on an empty stack.

Stack overflow: നിറഞ്ഞ stack-ലേക്ക് push ചെയ്യാൻ ശ്രമിക്കുമ്പോൾ ഉണ്ടാകുന്ന error അവസ്ഥ.

Stack underflow: ഒഴിഞ്ഞ (empty) stack-ൽ നിന്ന് pop ചെയ്യാൻ ശ്രമിക്കുമ്പോൾ ഉണ്ടാകുന്ന error അവസ്ഥ.

In the unit's algorithms the stack is an array `arr` with positions 1 to `MAXSTACK`, and `tos` (top of stack) holds the position of the topmost item; `tos = 0` means empty. Push first checks for overflow, then increments `tos` and stores the item. Pop first checks for underflow, then reads the item and decrements `tos`. Both take $O(1)$ time.

മലയാളത്തിൽ

ഒരു ബസ്സിൽ സീറ്റ് മുഴുവൻ നിറഞ്ഞിട്ടും ഒരാളെക്കൂടി ഇരുത്താൻ ശ്രമിക്കുന്നത് പോലെയാണ് overflow. ബസ്സിൽ ആരുമില്ലാത്തപ്പോൾ ഒരാളെ ഇറക്കാൻ നോക്കുന്നത് പോലെയാണ് underflow. അതുകൊണ്ട് push-ന് മുമ്പ് "നിറഞ്ഞോ?" എന്നും pop-ന് മുമ്പ് "ഒഴിഞ്ഞോ?" എന്നും ആദ്യം check ചെയ്യണം.

PUSH(value)

Step 1: if $\text{tos} \geq \text{MAXSTACK}$, print "Stack overflow" and exit

Step 2: $\text{tos} = \text{tos} + 1$

Step 3: $\text{arr}[\text{tos}] = \text{value}$

Step 4: exit

POP()

Step 1: if $\text{tos} = 0$, print "Stack underflow" and exit

Step 2: $\text{value} = \text{arr}[\text{tos}]; \text{tos} = \text{tos} - 1$

Step 3: return value

EXAMPLE

$\text{MAXSTACK} = 3$, $\text{tos} = 0$. Trace: push 10, push 20, push 30, push 40, pop, pop, pop, pop.

Operation	Check	arr[1 . . 3]	tos	Result
push 10	$0 \geq 3?$ no	10	1	ok
push 20	$1 \geq 3?$ no	10 20	2	ok
push 30	$2 \geq 3?$ no	10 20 30	3	ok
push 40	$3 \geq 3?$ yes	10 20 30	3	Stack overflow
pop	$\text{tos} = 0?$ no	10 20	2	returns 30
pop	$\text{tos} = 0?$ no	10	1	returns 20
pop	$\text{tos} = 0?$ no	empty	0	returns 10
pop	$\text{tos} = 0?$ yes	empty	0	Stack underflow

Exam tip: Order matters. In push, increment tos before storing (tos points to the top item); in pop, read before decrementing. Always write the overflow/underflow check as Step 1.

Implementation of stack using arrays

Array implementation of a stack: storing the stack's elements in a fixed-size array, with an integer index (top) marking the top of the stack.

Array implementation of a stack: stack-ലെ elements ഒരു fixed-size array-ൽ സൂക്ഷിക്കുകയും, top എന്ന integer index കൊണ്ട് stack-ന്റെ മുകൾഭാഗം അടയാളപ്പെടുത്തുകയും ചെയ്യുന്ന രീതി.

An array already stores an ordered list, so it is the easiest way to hold a stack. The drawback is that the array size must be declared in advance, so the stack's size is fixed. In Program 4.1 the array `stack[10]` holds at most 10 integers. Here `top` is the index of the **next free location** (not the top item): `push` stores at `stack[top]` and then does `++top`; `pop` does `--top` and then reads `stack[top]`. The menu offers `push`, `pop`, `show elements` and `exit`.

മലയാളത്തിൽ

Array ഒരു നിരയായി അടുക്കിയ പെട്ടികൾ പോലെയാണ്; അതിൽ stack വെക്കാൻ എളുപ്പമാണ്. പക്ഷേ പെട്ടികളുടെ എണ്ണം ആദ്യമേ തീരുമാനിക്കണം, പിന്നെ മാറ്റാൻ പറ്റില്ല. ഈ program-ൽ `top` അടുത്ത ഒഴിഞ്ഞ പെട്ടിയുടെ index ആണ്: `push` ചെയ്യുമ്പോൾ അവിടെ വെച്ചിട്ട് `top` ഒന്ന് കൂട്ടും, `pop` ചെയ്യുമ്പോൾ `top` ഒന്ന് കുറച്ചിട്ട് അവിടുന്ന് എടുക്കും.

`top` = number of elements = index of next free slot (0-based array)

Empty: `top == 0` Full: `top == 10`

`push`: `stack[top] = x; ++top` `pop`: `--top; x = stack[top]`

PROGRAM 4.1 (CORRECTED): STACK USING AN ARRAY

```

#include <stdio.h>

int choice, stack[10], top, element;

void menu();
void push();
void pop();
void showelements();

int main()
{
    choice = element = 1;
    top = 0;
    menu();
    return 0;
}

void menu()
{
    do {
        printf("Enter one of the following options:\n");
        printf("PUSH 1\nPOP 2\nSHOW ELEMENTS 3\nEXIT 4\n");
        if (scanf("%d", &choice) != 1)
            break;
        if (choice == 1)
            push();
        else if (choice == 2)
            pop();
        else if (choice == 3)
            showelements();
    } while (choice != 4);
}

void push()
{
    if (top <= 9) {
        printf("Enter the element to be pushed to stack:\n");
        scanf("%d", &element);
        stack[top] = element;
        ++top;
    } else {
        printf("Stack is full\n");
    }
}

void pop()
{
    if (top > 0) {
        --top;
        element = stack[top];
        printf("Popped element:%d\n", element);
    }
}

```

```

    } else {
        printf("Stack is empty\n");
    }
}

void showelements()
{
    if (top <= 0)
        printf("Stack is empty\n");
    else
        for (int i = 0; i < top; ++i)
            printf("%d\n", stack[i]);
}

```

Run: push 10, 20, 30; show; pop; show; exit (menu lines omitted):

```

Enter the element to be pushed to stack:
Enter the element to be pushed to stack:
Enter the element to be pushed to stack:
10
20
30
Popped element:30
10
20

```

Show elements prints from stack[0] upward, so the bottom item comes first.

Exam tip: Two fixes were made to IGNOU's Program 4.1. (1) void main() is not standard C; main must return int. (2) The unit's menu() calls itself again after every choice, so each operation adds a new function call on the system stack, and a non-numeric input makes scanf fail with the old choice kept, recursing forever until the program crashes. The corrected version uses a do ... while loop. Also note the program's top is the next free index (0-based), while the algorithm in Figure 4.3 uses a 1-based tos that points to the top item; both are correct, but do not mix them in one answer.

Implementation of stack using linked lists

Linked stack: a stack implemented as a singly linked list in which push inserts a node at the front and pop deletes the front node; the head pointer is the top of stack.

Linked stack: singly linked list ആയി implement ചെയ്ത stack; push മുനിൽ ഒരു node ചേർക്കുന്നു, pop മുനിൽ node delete ചെയ്യുന്നു; head pointer ആണ് top of stack.

An array stack cannot grow or shrink, so we reserve too much or too little space. A linked list removes this limit: each node holds data and a pointer next to the node below it, and nodes are created with `malloc` and released with `free` as needed. Push and pop both work at the front of the list, so both are $O(1)$. The pointer `tos` is passed as node `**` so the functions can change the caller's top pointer.

മലയാളത്തിൽ

ഒരു ചങ്ങലയിലെ കണ്ണികൾ പോലെയാണ് linked list: ഓരോ node-ലും data-യും അടുത്ത node-ന്റെ address-ഉം ഉണ്ട്. പുതിയ കണ്ണി എപ്പോഴും മുന്നിൽ ചേർക്കും (push), എടുക്കുന്നതും മുന്നിൽ നിന്നു തന്നെ (pop). ആവശ്യമുള്ളപ്പോൾ മാത്രം `malloc` കൊണ്ട് memory എടുക്കുന്നതിനാൽ size മുൻകൂട്ടി പറയേണ്ട; memory തീരുമ്പോൾ മാത്രമേ overflow വരൂ.

```
push: temp = new node; temp → data = item; temp → next = tos; tos = temp
```

```
pop: temp = tos; tos = tos → next; item = temp → data; free(temp)
```

```
Empty: tos == NULL
```

PROGRAM 4.2 (CORRECTED): STACK USING A LINKED LIST

```

#include <stdio.h>
#include <stdlib.h>

typedef struct node {
    int data;
    struct node *next;
} node;

void push(node **tos, int item)
{
    node *temp = (node *)malloc(sizeof(node));
    if (temp == NULL) {
        printf("\nError: Insufficient Memory Space");
        return;
    }
    temp->data = item;
    temp->next = *tos;
    *tos = temp;
}

/* returns 1 and stores the popped value in *item; returns 0 if empty */
int pop(node **tos, int *item)
{
    node *temp = *tos;
    if (temp == NULL)
        return 0;
    *tos = temp->next;
    *item = temp->data;
    free(temp);
    return 1;
}

void display(node *tos)
{
    node *temp = tos;
    if (temp == NULL) {
        printf("\nStack is empty");
        return;
    }
    while (temp != NULL) {
        printf("\n%d", temp->data);
        temp = temp->next;
    }
}

int main()
{
    int item, ch;
    char choice = 'y';
    node *p = NULL;
    do {

```

```

printf("\n*****MENU*****");
printf("\n1. To PUSH an element");
printf("\n2. To POP an element");
printf("\n3. To DISPLAY the elements of stack");
printf("\n4. Exit");
printf("\nEnter your choice:-");
if (scanf("%d", &ch) != 1)
    break;
switch (ch) {
case 1:
    printf("\nEnter an element which you want to push ");
    scanf("%d", &item);
    push(&p, item);
    break;
case 2:
    if (pop(&p, &item))
        printf("\nDeleted item is %d", item);
    else
        printf("\nStack is empty");
    break;
case 3:
    printf("\nThe elements of stack are");
    display(p);
    break;
case 4:
    exit(0);
}
printf("\nDo you want to run it again y/n ");
scanf(" %c", &choice);
} while (choice == 'y');
return 0;
}

```

Run: push 10, 20, 30; display; pop; display (menu and prompts omitted):

```

The elements of stack are
30
20
10
Deleted item is 30
The elements of stack are
20
10

```

Display walks from the top node to the last node, so the newest item (30) is printed first.

EXAMPLE: POINTER TRACE

Start with $p = \text{NULL}$. push 10: new node [10|NULL], $p \rightarrow 10$. push 20: new node [20|10], $p \rightarrow 20$. push 30: $p \rightarrow 30 \rightarrow 20 \rightarrow 10 \rightarrow \text{NULL}$. pop: temp = node 30, p moves to 20, 30 is returned and its node freed. List: $p \rightarrow 20 \rightarrow 10 \rightarrow \text{NULL}$.

Exam tip: IGNOU's Program 4.2 does not compile as printed. Fixes: (1) `typedef struct node {...};` gives no type name, so node is undefined; it must be `typedef struct node {...} node;`. (2) pop returns NULL (a pointer) from an int function, and a real value 0 could not be told apart from "empty"; the fix returns a success flag and passes the item back through a pointer. (3) `if (item!=NULL);` has a stray semicolon, so the message always printed. (4) `scanf ("%c",&choice)` reads the leftover newline, so the loop always stopped after one run; `" %c"` skips it. (5) `void main`, curly quotes, and the Turbo C-only `conio.h`, `clrscr()`, `getch()` were replaced with standard C. The message "Detected item" should read "Deleted item".

Array stack vs linked stack

Static vs dynamic stack: an array stack has a fixed size set at compile time; a linked stack grows and shrinks at run time, one node per element.

Static vs dynamic stack: array stack-ന്റെ size compile time-ൽ തന്നെ fixed ആണ്; linked stack run time-ൽ ഓരോ element-നും ഓരോ node വീതം വലുതാകുകയും ചെറുതാകുകയും ചെയ്യുന്നു.

This answers Check Your Progress 1, Q2. The linked representation is better when the maximum size is not known, because it never wastes unused slots and never overflows while memory is available. The array representation is simpler, needs no pointer per element, and gives slightly faster access, so it is fine when the maximum size is known.

മലയാളത്തിൽ

Array stack എണ്ണം നിശ്ചയിച്ച സീറ്റുകളുള്ള ഓട്ടോ പോലെ: സീറ്റ് ഒഴിഞ്ഞു കിടന്നാലും പാഴ്, നിറഞ്ഞാൽ പിന്നെ കയറ്റാൻ പറ്റില്ല (overflow). Linked stack ആവശ്യത്തിന് കോച്ച് കൂട്ടാവുന്ന ട്രെയിൻ പോലെ. പക്ഷേ ഓരോ node-നും ഒരു pointer-ന്റെ അധികം memory വേണം.

Point	Array stack	Linked stack
Size	Fixed, declared in advance	Dynamic, grows and shrinks
Memory use	Unused slots are wasted	Extra pointer in every node
Overflow	When top reaches the array size	Only when malloc fails (memory exhausted)
Underflow	top == 0 (or -1)	tos == NULL
Push / pop time	O(1)	O(1)
Code	Simpler	Needs pointers, malloc, free

EXAMPLE

A program needs a stack whose size could be 5 on one day and 50,000 on another. Which representation? A linked stack. An array would have to be declared with 50,000 slots and waste almost all of them on small days; the linked stack uses exactly one node per item.

Exam tip: For "discuss overflow and underflow": underflow can happen in both representations (pop on empty); overflow is the array's real problem, while a linked stack overflows only when the heap has no free memory.

Two stacks in one array

Two stacks in one array: stack A starts at the left end and grows right, stack B starts at the right end and grows left; overflow occurs only when their tops meet.

Two stacks in one array: stack A ഇടത്തെ അറ്റത്തു നിന്ന് വലത്തോട്ടും, stack B വലത്തെ അറ്റത്തു നിന്ന് ഇടത്തോട്ടും വളരുന്നു; രണ്ടിന്റെയും top കൂട്ടിമുട്ടുമ്പോൾ മാത്രമേ overflow ഉണ്ടാകൂ.

If stacks A and B get separate arrays of sizes n_1 and n_2 , A overflows when it holds more than n_1 items even if B's array is almost empty. Using one array of size $n = n_1 + n_2$, with the two bottoms at the two fixed ends ($X(1)$ and $X(n)$), the free space in the middle is shared. Overflow now happens only when A and B together hold more than n items.

മലയാളത്തിൽ

ഒരു നീണ്ട ബെഞ്ചിന്റെ രണ്ടറ്റത്തു നിന്നും രണ്ടു ടീമുകൾ ഇരുന്നു തുടങ്ങുന്നു എന്ന് കരുതുക. നടുവിലെ ഒഴിഞ്ഞ സ്ഥലം ആർക്കു വേണമെങ്കിലും ഉപയോഗിക്കാം. രണ്ടു ടീമും നടുവിൽ മുട്ടുമ്പോൾ മാത്രമേ ബെഞ്ച് "full" ആകൂ. ഓരോ ടീമിനും വേറെ ബെഞ്ച് കൊടുത്താൽ, ഒന്നിൽ സ്ഥലം ഒഴിഞ്ഞു കിടന്നാലും മറ്റേതിൽ ഇടം കിട്ടില്ല.

```
Array arr[0..n-1]; initially topA = -1, topB = n
pushA: arr[++topA] = x   pushB: arr[--topB] = x
Overflow (for either): topA + 1 == topB
A empty: topA == -1    B empty: topB == n
```

EXAMPLE

Array of size 6. Push 1, 2, 3, 4 onto A, then 9, 8 onto B, then try 7 onto B.

```
#include <stdio.h>

#define SIZE 6

int arr[SIZE];
int topA = -1, topB = SIZE;    /* A grows right, B grows left */

void pushA(int x)
{
    if (topA + 1 == topB) { printf("Overflow\n"); return; }
    arr[++topA] = x;
}

void pushB(int x)
{
    if (topB - 1 == topA) { printf("Overflow\n"); return; }
    arr[--topB] = x;
}

int main()
{
    pushA(1); pushA(2); pushA(3); pushA(4);    /* A uses 4 cells */
    pushB(9); pushB(8);                        /* B uses 2 cells */
    pushB(7);                                  /* array is full */
    printf("topA = %d, topB = %d\n", topA, topB);
    return 0;
}
```

```
Overflow
topA = 3, topB = 4
```

After four pushes on A, $\text{topA} = 3$; after two on B, $\text{topB} = 4$. Now $\text{topA} + 1 = 4 = \text{topB}$, so all 6 cells are used and pushing 7 overflows. With two separate arrays of size 3 each, A would already have overflowed at its 4th item.

Multiple (n) stacks in one array

Multiple stacks: n stacks kept in one array $X(1:m)$ by dividing it into n segments; for stack i , $BM(i)$ is one less than its bottom position and $TM(i)$ is its top position.

Multiple stacks: $X(1:m)$ എന്ന ഒരു array-നെ n segments ആയി ഭാഗിച്ച് n stack-കൾ സൂക്ഷിക്കുന്ന രീതി; stack i -ന് $BM(i)$ അതിന്റെ bottom position-നെക്കാൾ ഒന്ന് കുറവും, $TM(i)$ അതിന്റെ top position-ഉം ആണ്.

With more than two stacks the two-ends trick fails, because a 1-D array has only two fixed points, $X(1)$ and $X(m)$, and every stack needs a fixed point for its bottom. So $X(1:m)$ is split into n segments: in proportion to the expected sizes if known, otherwise into equal segments. Stack i is empty exactly when $BM(i) = TM(i)$. Stack i is full when its top reaches the start of the next stack's segment, $TM(i) = BM(i + 1)$.

മലയാളത്തിൽ

ഒരു നീണ്ട അലമാരയിലെ ഒരു തട്ടിനെ n കള്ളികളാക്കി ഓരോ കട്ടിക്കും ഓരോ കള്ളി കൊടുക്കുന്നതു പോലെ. $BM(i)$ കള്ളിയുടെ തുടക്കത്തിന് തൊട്ടു മുമ്പുള്ള സ്ഥാനം, $TM(i)$ ഇപ്പോഴത്തെ ഏറ്റവും മുകളിലെ പുസ്തകം. $BM(i) = TM(i)$ ആണെങ്കിൽ കള്ളി ഒഴിഞ്ഞത്; $TM(i)$ അടുത്ത കള്ളിയുടെ BM -ൽ എത്തിയാൽ കള്ളി നിറഞ്ഞു.

Initial values (equal segments): $BM(i) = TM(i) = \lfloor m/n \rfloor \times (i - 1)$, $1 \leq i \leq n$

Also set $BM(n + 1) = m$ (end of X), so the full test works for the last stack

Stack i empty $\Rightarrow TM(i) = BM(i)$ Stack i full $\Rightarrow TM(i) = BM(i + 1)$

ADD(i, e)

Step 1: if $TM(i) = BM(i + 1)$, print "Stack is full" and exit

Step 2: $TM(i) \leftarrow TM(i) + 1$; $X(TM(i)) \leftarrow e$

Step 3: exit

DELETE(i, e)

Step 1: if $TM(i) = BM(i)$, print "Stack is empty" and exit

Step 2: $e \leftarrow X(TM(i))$; $TM(i) \leftarrow TM(i) - 1$

Step 3: exit

EXAMPLE 1

$m = 20$, $n = 4$. Find the initial BM and TM and each stack's positions.

$\lfloor 20/4 \rfloor = 5$. $BM(1) = 0$, $BM(2) = 5$, $BM(3) = 10$, $BM(4) = 15$, and $BM(5) = 20$. $TM(i) = BM(i)$ at the start.

Stack 1 uses $X(1..5)$, stack 2 $X(6..10)$, stack 3 $X(11..15)$, stack 4 $X(16..20)$.

EXAMPLE 2 (C PROGRAM OF ADD AND DELETE)

$m = 10, n = 3: \lfloor 10/3 \rfloor = 3$, so $BM = 0, 3, 6$ and $BM(4) = 10$. Stack 1 has $X(1..3)$, stack 2 $X(4..6)$, stack 3 gets the leftover $X(7..10)$.

```

#include <stdio.h>

#define M 10          /* size of array X, used as X[1..M] */
#define N 3           /* number of stacks */

int X[M + 1];
int BM[N + 2], TM[N + 1]; /* BM[N+1] = M marks the end of X */

void init(void)
{
    int i;
    for (i = 1; i <= N; i++)
        BM[i] = TM[i] = (M / N) * (i - 1);
    BM[N + 1] = M;
}

void add(int i, int e)
{
    if (TM[i] == BM[i + 1]) {
        printf("Stack %d is full\n", i);
        return;
    }
    TM[i] = TM[i] + 1;
    X[TM[i]] = e;
}

int delete(int i, int *e)
{
    if (TM[i] == BM[i]) {
        printf("Stack %d is empty\n", i);
        return 0;
    }
    *e = X[TM[i]];
    TM[i] = TM[i] - 1;
    return 1;
}

int main()
{
    int e, k;
    init();
    for (k = 1; k <= N; k++)
        printf("Stack %d: BM = %d, TM = %d\n", k, BM[k], TM[k]);
    add(1, 11); add(1, 12); add(1, 13);
    add(1, 14); /* stack 1 has only X[1..3] */
    add(3, 31);
    if (delete(1, &e)) printf("Deleted %d from stack 1\n", e);
    delete(2, &e); /* stack 2 is still empty */
    printf("TM(1) = %d, TM(3) = %d\n", TM[1], TM[3]);
}

```

```
return 0;  
}
```

```
Stack 1: BM = 0, TM = 0  
Stack 2: BM = 3, TM = 3  
Stack 3: BM = 6, TM = 6  
Stack 1 is full  
Deleted 13 from stack 1  
Stack 2 is empty  
TM(1) = 2, TM(3) = 7
```

The 4th ADD to stack 1 finds $TM(1) = 3 = BM(2)$, so it reports full even though stacks 2 and 3 have free space. This is the weakness of fixed segments: a real system would then shift neighbouring stacks to make room.

Exam tip: The extracted unit prints the range as " $1 < i < n$ "; it should be $1 \leq i \leq n$ (all n stacks, including the first and last). The ADD test $TM(i) = BM(i + 1)$ needs $BM(n + 1) = m$ defined for the last stack; the unit does not say this, so state it in your answer.

Applications; infix, prefix and postfix notation

Infix: the operator is written between its operands ($A + B$). **Prefix (Polish):** the operator is written before its operands ($+ A B$). **Postfix (reverse Polish):** the operator is written after its operands ($A B +$).

Infix: operator രണ്ട് operands-ന്റെ നടുവിൽ എഴുതുന്നു ($A + B$). **Prefix (Polish):** operator operands-ന് മുമ്പ് എഴുതുന്നു ($+ A B$). **Postfix (reverse Polish):** operator operands-ന് ശേഷം എഴുതുന്നു ($A B +$).

Stacks are used to evaluate arithmetic expressions, to convert between infix, prefix and postfix forms, and inside the computer whenever (recursive) functions are called. Prefix and postfix are called Polish notations; they need **no parentheses** and no precedence rules, because the order of the operators already fixes the order of evaluation. A computer evaluates an infix expression in two steps, each using a stack: (1) convert infix to postfix, (2) evaluate the postfix expression.

മലയാളത്തിൽ

നമ്മൾ സാധാരണ എഴുതുന്ന $A + B$ ആണ് infix. കമ്പ്യൂട്ടറിന് brackets-ഉം precedence-ഉം നോക്കി കണക്കു കൂട്ടാൻ ബുദ്ധിമുട്ടാണ്. അതുകൊണ്ട് ആദ്യം അതിനെ postfix ($A B +$) ആക്കി മാറ്റും, പിന്നെ ഇടത്തു നിന്ന് വലത്തോട്ട് ഒരു stack ഉപയോഗിച്ച് നേരെ value കണ്ടുപിടിക്കും. രണ്ടു ഘട്ടത്തിലും stack ആണ് സഹായി.

Precedence (high $\rightarrow$ low): $\wedge$ then $\ast /$ then $+ -$
 $\ast / + -$ are left-associative: $A - B + C = (A - B) + C$
 $\wedge$ is right-associative: $A \wedge B \wedge C = A \wedge (B \wedge C)$

EXAMPLE: HAND CONVERSION BY BRACKETS

Write $(A + B) \ast C$ in all three forms.

Fully bracket: $((A + B) \ast C)$. For postfix, move each operator just after its right bracket: $((A B +) C \ast) \rightarrow A B + C \ast$. For prefix, move each operator just before its left bracket: $(\ast (+ A B) C) \rightarrow \ast + A B C$. The order of operands A, B, C never changes; only the operators move.

Infix	Postfix	Prefix
$A + B \ast C$	$A B C \ast +$	$+ A \ast B C$
$(A + B) \ast C$	$A B + C \ast$	$\ast + A B C$
$A - B + C$	$A B - C +$	$+ - A B C$
$A \wedge B \wedge C$	$A B C \wedge \wedge$	$\wedge A \wedge B C$
$(A + B) \ast (C - D)$	$A B + C D - \ast$	$\ast + A B - C D$

Exam tip: Check any conversion by counting: the operands keep their left-to-right order in all three forms, and in postfix every operator appears only after both of its operands.

Converting infix to postfix

Infix-to-postfix conversion: scanning the infix expression left to right, sending operands straight to the output and holding operators and '(' on a stack until precedence rules allow them to be output.

Infix-to-postfix conversion: infix expression ഇടത്തു നിന്ന് വലത്തോട്ട് scan ചെയ്ത്, operands നേരെ output-ലേക്ക് അയക്കുകയും, precedence നിയമം അനുവദിക്കുന്നത് വരെ operators-ഉം '('-ഉം ഒരു stack-ൽ വെക്കുകയും ചെയ്യുന്ന രീതി.

Operands never wait; operators wait on the stack because a later operator may bind tighter. When a new operator arrives, first pop to the output every operator on top of the stack with higher precedence (or equal precedence, for left-associative operators), stopping at '('. A ')' pops everything back to the matching '('. At the end, pop whatever is left.

മലയാളത്തിൽ

Operands (A, B, ...) കണ്ടാൽ ഉടനെ output-ൽ എഴുതാം. Operator വന്നാൽ അതിനെ stack-ൽ കാത്തുനിർത്തണം, കാരണം പിന്നാലെ വരുന്ന operator-ന് കൂടുതൽ മുൻഗണന ഉണ്ടാകാം. പുതിയ operator stack-ൽ കയറും മുൻ, അതിനെക്കാൾ വലിയതോ തുല്യമോ ആയ മുൻഗണനയുള്ളവയെ മുകളിൽ നിന്ന് ഇറക്കി output-ൽ എഴുതണം. ')' കണ്ടാൽ '(' വരെയുള്ളതെല്ലാം ഇറക്കണം.

Scan the infix expression left to right:

1. Operand → append to output
2. '(' → push
3. ')' → pop and output until '('; discard the '('
4. Operator op → while top is an operator with ($\text{prec} > \text{prec}(\text{op})$) or ($\text{prec} = \text{prec}(\text{op})$ and op is left-associative): pop and output; then push op
5. End of input → pop and output everything left on the stack

EXAMPLE 1

Convert $A + B * C$.

Symbol	Stack	Postfix	Reason
A	empty	A	operand
+	+	A	stack empty, push
B	+	A B	operand
*	+ *	A B	* higher than +, push
C	+ *	A B C	operand
end	empty	A B C * +	pop all

EXAMPLE 2 (LEFT ASSOCIATIVITY)

Convert $A - B + C$.

Symbol	Stack	Postfix	Reason
A	empty	A	operand
-	-	A	push
B	-	A B	operand
+	+	A B -	- has equal precedence, left-associ: pop -, push +
C	+	A B - C	operand
end	empty	A B - C +	pop all

For $A \wedge B \wedge C$ the second $\wedge$ does not pop the first (right-associative), so the result is $A B C \wedge \wedge$.

EXAMPLE 3 (EXAM-LENGTH)

Convert $A + (B * C - (D / E ^ F) * G) * H$.

Symbol	Stack	Postfix
A	empty	A
+	+	A
(	+ (	A
B	+ (	A B
*	+ (*	A B
C	+ (*	A B C
-	+ (-	A B C *
(	+ (- (	A B C *
D	+ (- (	A B C * D
/	+ (- (/	A B C * D
E	+ (- (/	A B C * D E
^	+ (- (/ ^	A B C * D E
F	+ (- (/ ^	A B C * D E F
)	+ (-	A B C * D E F ^ /
*	+ (- *	A B C * D E F ^ /
G	+ (- *	A B C * D E F ^ / G
)	+	A B C * D E F ^ / G * -
*	+ *	A B C * D E F ^ / G * -
H	+ *	A B C * D E F ^ / G * - H
end	empty	A B C * D E F ^ / G * - H * +

Answer: $A B C * D E F ^ / G * - H * +$

EXAMPLE 4: C PROGRAM

```

#include <stdio.h>
#include <ctype.h>

char stack[50];
int top = -1;

void push(char c) { stack[++top] = c; }
char pop(void)    { return stack[top--]; }

int prec(char c)
{
    if (c == '^') return 3;
    if (c == '*' || c == '/') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;                /* '(' */
}

int main()
{
    char infix[50], postfix[50];
    int i, k = 0;
    char c;

    scanf("%49s", infix);
    for (i = 0; infix[i] != '\0'; i++) {
        c = infix[i];
        if (isdigit(c))
            postfix[k++] = c;          /* operand: output */
        else if (c == '(')
            push(c);
        else if (c == ')') {
            while (stack[top] != '(')
                postfix[k++] = pop();
            pop();                    /* discard '(' */
        } else {                      /* operator */
            while (top != -1 && stack[top] != '(' &&
                (prec(stack[top]) > prec(c) ||
                 (prec(stack[top]) == prec(c) && c != '^')))
                postfix[k++] = pop();
            push(c);
        }
    }
    while (top != -1)
        postfix[k++] = pop();
    postfix[k] = '\0';
    printf("Postfix: %s\n", postfix);
    return 0;
}

```

Inputs $A+(B*C-(D/E^F)*G)*H$, $(A+B)*(C-D)$ and A^B^C give:

Postfix: $ABC*DEF^*/G*-H*+$
Postfix: $AB+CD-*$
Postfix: $ABC^^$

Exam tip: Parentheses never appear in the postfix output. The most common mistake is forgetting to pop an equal-precedence operator ($A - B + C$ must give $A B - C +$, not $A B C + -$).

Converting infix to prefix

Infix-to-prefix conversion: scanning the infix expression right to left with an operator stack (the mirror of the postfix method) and finally reversing the output.

Infix-to-prefix conversion: ഒരു operator stack ഉപയോഗിച്ച് infix expression വലത്തു നിന്ന് ഇടത്തോട്ട് scan ചെയ്ത് (postfix രീതിയുടെ കണ്ണാടി രൂപം), അവസാനം output തിരിച്ചിടുന്ന രീതി.

Scan from the right end. Operands go to the output. Here ')' plays the role of '(' (push it), and '(' pops back to the matching ')'. On an operator, pop only operators of strictly higher precedence (for '^', also equal), then push. At the end pop everything, and reverse the output string to get the prefix form.

മലയാളത്തിൽ

Postfix-ന്റെ അതേ രീതി, പക്ഷേ വലത്തു നിന്ന് ഇടത്തോട്ട്, brackets-ന്റെ റോൾ മാറ്റം, അവസാനം കിട്ടിയ string മുകളിൽ തിരിച്ച് എഴുതണം. ഒരു പ്രധാന വ്യത്യാസം: ഇവിടെ തുല്യ precedence ഉള്ള $+ - * /$ operators-നെ pop ചെയ്യരുത്, വലിയ precedence ഉള്ളവയെ മാത്രം. അല്ലെങ്കിൽ left-associativity തെറ്റും.

Scan the infix expression right to left:

1. Operand $\rightarrow$ append to output
2. ')' $\rightarrow$ push
3. '(' $\rightarrow$ pop and output until ')'; discard the ')'
4. Operator op $\rightarrow$ while top has $\text{prec} > \text{prec}(op)$ (or $= \text{prec}(op)$ when op is '^'): pop and output; then push op
5. End $\rightarrow$ pop everything; then **reverse** the output

EXAMPLE 1

Convert $A + B * C - D / E$ to prefix.

Symbol (right to left)	Stack	Output
E	empty	E
/	/	E
D	/	E D
-	-	E D /
C	-	E D / C
*	- *	E D / C
B	- *	E D / C B
+	- +	E D / C B *
A	- +	E D / C B * A
end	empty	E D / C B * A + -

At '+' the top '-' has equal precedence, so it is not popped. Reverse: $- + A * B C / D E$. Check by brackets: $((A + (B * C)) - (D / E)) \rightarrow - + A * B C / D E$. ✓

EXAMPLE 2 (WITH BRACKETS)

Convert $A * (B + C) / D$ to prefix.

Symbol (right to left)	Stack	Output
D	empty	D
/	/	D
)	/)	D
C	/)	D C
+	/) +	D C
B	/) +	D C B
(	/	D C B +
*	/ *	D C B +
A	/ *	D C B + A
end	empty	D C B + A * /

Reverse: $/ * A + B C D$. Check: $((A * (B + C)) / D) \rightarrow / * A + B C D$. ✓

EXAMPLE 3

Convert $(A + B) * (C - D)$ to prefix.

Right to left: ')' push; D out; '-' push; C out; '(' pops '-' → output D C -; '*' push; ')' push; B out; '+' push; A out; '(' pops '+' → D C - B A +; end pops '*' → D C - B A + *. Reverse: $* + A B - C D$.

Exam tip: For short expressions, the bracket method (fully bracket, move each operator to its left bracket) is fastest and safest. Use the stack table when the question asks for the stack contents.

Evaluating a postfix expression

Postfix evaluation: scanning the postfix expression left to right, pushing operands on a stack and, at each operator, popping two operands, applying the operator and pushing the result; the final stack item is the value.

Postfix evaluation: postfix expression ഇടത്തു നിന്ന് വലത്തോട്ട് scan ചെയ്ത്, operands stack-ൽ push ചെയ്യുകയും, ഓരോ operator വരുമ്പോൾ രണ്ട് operands pop ചെയ്ത് operation ചെയ്ത് ഫലം push ചെയ്യുകയും ചെയ്യുന്ന രീതി; അവസാനം stack-ൽ ശേഷിക്കുന്നതാണ് value.

This is the second step of expression evaluation and needs only an operand stack: no precedence and no brackets. When an operator arrives, the first value popped is the **right** operand and the second value popped is the **left** operand. A valid postfix expression leaves exactly one value on the stack.

മലയാളത്തിൽ

സംഖ്യ കണ്ടാൽ stack-ൽ വെക്കുക. Operator കണ്ടാൽ മുകളിലെ രണ്ടെണ്ണം എടുത്ത് കണക്കു ചെയ്ത് ഉത്തരം തിരികെ വെക്കുക. ശ്രദ്ധിക്കേണ്ടത്: ആദ്യം pop ചെയ്യുന്നത് വലതുവശത്തെ operand ആണ്. $12\ 4\ /\$ ൽ ആദ്യം 4 പുറത്തു വരും, പിന്നെ 12; ഉത്തരം $12\ /\ 4 = 3$, $4\ /\ 12$ അല്ല.

For each token, left to right:

operand $\rightarrow$ push

operator op $\rightarrow$ b = pop(); a = pop(); push(a op b)

End $\rightarrow$ value = pop() (the stack must then be empty)

EXAMPLE 1

Evaluate $5\ 6\ 2\ +\ *\ 12\ 4\ /\ -$.

Token	Action	Stack (bottom → top)
5	push	5
6	push	5, 6
2	push	5, 6, 2
+	$6 + 2 = 8$	5, 8
*	$5 * 8 = 40$	40
12	push	40, 12
4	push	40, 12, 4
/	$12 / 4 = 3$	40, 3
-	$40 - 3 = 37$	37

Value = 37. (Infix: $5 * (6 + 2) - 12 / 4 = 40 - 3 = 37 \checkmark$)

EXAMPLE 2

Evaluate $6\ 2\ 3\ +\ -\ 3\ 8\ 2\ /\ +\ * \ 2\ ^\ 3\ +$.

Token	Action	Stack (bottom → top)
6, 2, 3	push each	6, 2, 3
+	$2 + 3 = 5$	6, 5
-	$6 - 5 = 1$	1
3, 8, 2	push each	1, 3, 8, 2
/	$8 / 2 = 4$	1, 3, 4
+	$3 + 4 = 7$	1, 7
*	$1 * 7 = 7$	7
2	push	7, 2
^	$7 ^\ 2 = 49$	49
3	push	49, 3
+	$49 + 3 = 52$	52

Value = 52.

EXAMPLE 3: C PROGRAM (SINGLE-DIGIT OPERANDS)

```
#include <stdio.h>
#include <ctype.h>

int stack[50];
int top = -1;

void push(int x) { stack[++top] = x; }
int pop(void)    { return stack[top--]; }

int main()
{
    char exp[50];
    int i, a, b;

    scanf("%49s", exp);                /* single-digit operands, e.g. 234*+5- */
    for (i = 0; exp[i] != '\0'; i++) {
        if (isdigit(exp[i]))
            push(exp[i] - '0');
        else {
            b = pop();                  /* first pop = right operand */
            a = pop();
            switch (exp[i]) {
                case '+': push(a + b); break;
                case '-': push(a - b); break;
                case '*': push(a * b); break;
                case '/': push(a / b); break;
            }
        }
    }
    printf("Value = %d\n", pop());
    return 0;
}
```

Input 234*+5- (that is, $2 + 3 * 4 - 5$):

Value = 9

Trace: push 2, 3, 4; * gives 12 → [2, 12]; + gives 14 → [14]; push 5; - gives 14 - 5 = 9.

Exam tip: Check Your Progress 2 asks "_____ are evaluated by stacks": the unit's answer is postfix expressions (Polish notations in general). Swapping the two popped operands is the classic error for - and /.

Stacks and recursive function calls

Call stack: the system stack on which each function call pushes a frame (parameters, local variables, return address) that is popped when the call returns.

Call stack: ഓരോ function call-ഉം ഒരു frame (parameters, local variables, return address) push ചെയ്യുകയും, call return ചെയ്യുമ്പോൾ അത് pop ചെയ്യുകയും ചെയ്യുന്ന system stack.

A recursive function calls itself before the earlier call has finished, so every pending call must remember where it was. The computer pushes a frame for each call; the last call made is the first to finish (LIFO), and its frame is popped as it returns its value to the call below. Too deep a recursion fills this stack and causes a run-time stack overflow.

മലയാളത്തിൽ

ഒരു ഓഫീസിൽ ഫയൽ ഒരാളിൽ നിന്ന് അടുത്തയാളിലേക്ക് "ഇത് നോക്കിയിട്ട് തിരിച്ചു തരണം" എന്നു പറഞ്ഞ് കൈമാറുന്നതു പോലെ. അവസാനം ഫയൽ കിട്ടിയ ആൾ ആദ്യം ജോലി തീർത്ത് തിരികെ കൊടുക്കുന്നു; അങ്ങനെ പിന്നോട്ട്. ഓരോരുത്തരും എവിടെ നിർത്തി എന്ന് ഓർക്കുന്നത് stack-ലെ frame-കളിലാണ്.

$\text{fact}(n) = 1$ if $n = 0$; $n \times \text{fact}(n - 1)$ otherwise

EXAMPLE

Show the call stack for fact(3).

Event	Call stack (bottom → top)
call fact(3)	fact(3)
call fact(2)	fact(3), fact(2)
call fact(1)	fact(3), fact(2), fact(1)
call fact(0)	fact(3), fact(2), fact(1), fact(0)
fact(0) returns 1	fact(3), fact(2), fact(1)
fact(1) returns $1 \times 1 = 1$	fact(3), fact(2)
fact(2) returns $2 \times 1 = 2$	fact(3)
fact(3) returns $3 \times 2 = 6$	empty

The calls were made in the order 3, 2, 1, 0 and finished in the order 0, 1, 2, 3: LIFO.

Exam tip: Check Your Progress 2 Q3: "Stack is used whenever a recursive function is called." (In fact every function call uses the call stack; recursion is where it matters most.)

Quick recap

- A stack inserts and deletes only at the top: LIFO / FILO, also called a pile or push-down list.
- Push inserts at the top; pop deletes from the top; peek reads the top without removing it.
- Overflow = push on a full stack; underflow = pop on an empty stack. Check these first.
- Algorithm (1-based): push is $\text{tos} = \text{tos} + 1$ then $\text{arr}[\text{tos}] = \text{value}$; pop is $\text{value} = \text{arr}[\text{tos}]$ then $\text{tos} = \text{tos} - 1$.
- Array stack: simple, fixed size; Program 4.1 uses top as the next free index (0 = empty, 10 = full).
- Linked stack: push and pop at the front of a singly linked list; empty when $\text{tos} == \text{NULL}$; no fixed size.
- Both representations give $O(1)$ push and pop.
- Two stacks in one array grow from opposite ends; overflow only when $\text{topA} + 1 == \text{topB}$.
- n stacks in $X(1:m)$: $\text{BM}(i) = \text{TM}(i) = \lfloor m/n \rfloor (i - 1)$; empty iff $\text{TM}(i) = \text{BM}(i)$; full iff $\text{TM}(i) = \text{BM}(i + 1)$, with $\text{BM}(n + 1) = m$.
- Infix $A + B$, prefix $+ A B$, postfix $A B +$; Polish forms need no brackets.
- Infix $\rightarrow$ postfix: scan left to right, operands out, operators wait on the stack; pop higher or equal precedence (not equal for $^$).
- Infix $\rightarrow$ prefix: scan right to left, pop only strictly higher precedence, then reverse the output.

- Postfix evaluation: operands pushed; on an operator, b = pop, a = pop, push a op b.
- Every function call, especially a recursive one, uses the system call stack.

Check yourself

Q. 1. True or False: (a) Stacks are sometimes called FIFO lists. (b) A stack allows push and pop from both ends. (c) TOS gives the bottommost element of the stack.

(a) False: a stack is LIFO (FIFO is a queue). (b) False: push and pop happen only at the top. (c) False: TOS (top of stack) gives the topmost element.

Q. 2. Why is the linked list representation of a stack better than the array representation?

An array's size is fixed when it is declared, so we reserve either too much space (waste) or too little (overflow). A linked stack creates a node with `malloc` on each push and frees it on each pop, so it uses exactly as much memory as needed and overflows only when memory is exhausted. Push and pop are still $O(1)$ at the front of the list. The cost is one extra pointer per node.

Q. 3. Discuss the underflow and overflow problems in stacks.

Underflow: a pop (or peek) on an empty stack; there is no element to remove. Test `tos = 0` (array) or `tos == NULL` (linked list) before popping. Overflow: a push when the stack is full; in an array stack this happens when `tos` reaches `MAXSTACK`, in a linked stack only when `malloc` returns `NULL`. Test before pushing and report the error instead of writing outside the array.

Q. 4. An empty array stack has `MAXSTACK = 4`. Perform push 3, push 7, pop, push 1, push 6, push 2, push 8, pop. What is popped and what is left?

push 3 → [3]; push 7 → [3, 7]; pop → 7, [3]; push 1 → [3, 1]; push 6 → [3, 1, 6]; push 2 → [3, 1, 6, 2] (now full, `tos = 4`); push 8 → **overflow**, stack unchanged; pop → 2, [3, 1, 6]. Popped: 7 and 2. Left: 3, 1, 6 with 6 at the top (`tos = 3`).

Q. 5. Multiple stacks can be implemented using _____. Stack is used whenever a _____ function is called.

Arrays (or linked lists / pointers). Recursive.

Q. 6. X(1:24) holds $n = 3$ stacks in equal segments. Give the initial BM and TM, and the condition for stack 2 to be full.

$\lfloor 24/3 \rfloor = 8$, so $BM(1) = TM(1) = 0$, $BM(2) = TM(2) = 8$, $BM(3) = TM(3) = 16$, and $BM(4) = 24$. Stack 1 uses X(1..8), stack 2 X(9..16), stack 3 X(17..24). Stack 2 is full when $TM(2) = BM(3) = 16$.

Q. 7. Convert $(A + B) * C - D / E$ to postfix, showing the stack.

(push; A out; + push $\rightarrow$ (+; B out;) pops + $\rightarrow$ A B +; * push; C out $\rightarrow$ A B + C; - pops * (higher) and pushes - $\rightarrow$ A B + C *; D out; / pushes over - $\rightarrow$ stack - /; E out; end pops / then -.

Answer: **A B + C * D E / -**.

Q. 8. Convert $(A + B) * C - D / E$ to prefix.

Fully bracket: $((A + B) * C) - (D / E)$. Move each operator to its left bracket: $- * + A B C / D E$. Answer: $- * + A B C / D E$. (The right-to-left stack method gives output $E D / C B A + * -$, which reversed is the same.)

Q. 9. Evaluate the postfix expression $8\ 2\ 3\ * -\ 4\ 2\ /\ +$.

push 8, 2, 3 $\rightarrow$ [8, 2, 3]; * $\rightarrow 2 * 3 = 6$, [8, 6]; - $\rightarrow 8 - 6 = 2$, [2]; push 4, 2 $\rightarrow$ [2, 4, 2]; / $\rightarrow 4 / 2 = 2$, [2, 2]; + $\rightarrow 2 + 2 = 4$. Value = **4**.

Q. 10. Why do computers convert infix to postfix before evaluating?

Infix needs precedence rules and brackets to decide the order of operations, so the evaluator would have to look ahead. Postfix has no brackets and the operators already appear in evaluation order, so a single left-to-right scan with one stack evaluates it. Both steps (conversion and evaluation) use a stack.

UNIT 5

Queues

ഒരു അറ്റത്ത് (rear) ചേർക്കുകയും മറ്റേ അറ്റത്ത് (front) നിന്ന് എടുക്കുകയും ചെയ്യുന്ന FIFO data structure ആയ queue ആണ് ഈ unit-ൽ. Simple queue, multiple queues, circular queue, dequeue എന്നിവയും അവയുടെ array, linked list implementations-ഉം C programs സഹിതം പഠിക്കുന്നു.

Queue as an abstract data type

Front, rear and the add/delete algorithms

Array implementation of a queue

Linked list implementation of a queue

Multiple queues

Circular queue: idea and algorithms

Array implementation of a circular queue

Linked list implementation of a circular queue

Dequeue (double ended queue)

Array implementation of a dequeue

Linked list implementation of a dequeue

Queue as an abstract data type

Queue: a linear data structure in which elements are added at one end (rear) and deleted from the other end (front), so it follows First In First Out (FIFO) order.

Abstract Data Type (ADT): a description of a data structure by its values and operations, without specifying how it is implemented.

Queue: ഒരു അറ്റത്ത് (rear) elements ചേർക്കുകയും മറ്റേ അറ്റത്ത് (front) നിന്ന് delete ചെയ്യുകയും ചെയ്യുന്ന linear data structure; അതുകൊണ്ട് അത് First In First Out (FIFO) order പാലിക്കുന്നു.

Abstract Data Type (ADT): ഒരു data structure-നെ അതിന്റെ values-ഉം operations-ഉം വഴി വിവരിക്കുന്നത്; എങ്ങനെ implement ചെയ്യുന്നു എന്ന് പറയാതെ.

A queue is like the line of customers at a service counter: the one who came first is served first. IGNOU defines the queue ADT by its operations only. Any program written using these operations works whether the queue is built with an array or a linked list; only the efficiency differs.

മലയാളത്തിൽ

Supplyco-യിലോ ബസ് സ്റ്റോപ്പിലോ ഉള്ള വരി ഓർക്കുക: ആദ്യം വന്നയാൾ ആദ്യം പോകുന്നു, പുതിയ ആൾ പിറകിൽ നിൽക്കുന്നു. ഇതാണ് FIFO. ADT എന്നാൽ "ഈ queue-ൽ എന്തൊക്കെ ചെയ്യാം" എന്ന list മാത്രം; array ആണോ linked list ആണോ ഉള്ളിൽ എന്നത് പിന്നത്തെ കാര്യം.

createQueue(Q): creates an empty queue Q

isEmpty(Q): true if Q exists and is empty, false otherwise

addQueue(Q, item): adds item at the rear of Q

deleteQueue(Q, item): deletes an item from Q

next(Q): removes the least recently added item and returns it

Axioms: isEmpty(createQueue(Q)) = true; deleteQueue(createQueue(Q)) = error

EXAMPLE

Starting from an empty queue, perform addQueue(A), addQueue(B), addQueue(C), next, addQueue(D), next. What is returned and what is left?

After three adds: front $\rightarrow$ A B C $\leftarrow$ rear.

next returns A (least recently added). Queue: B C.

addQueue(D): B C D.

next returns B. Queue left: C D (front = C, rear = D).

EXAMPLE 2

Give two uses of a queue in a computer.

(1) Print jobs sent to a printer: the job fired first is printed first. (2) CPU job scheduling: processes wait in a ready queue for their turn.

Exam tip: isEmpty is needed because calling next (or deleteQueue) on an empty queue is an error, called underflow. The ADT has no "isFull", because fullness depends on the actual size chosen in a program; adding to a full array queue is overflow.

Front, rear and the add/delete algorithms

Front: the end of the queue from which elements are deleted. **Rear:** the end of the queue at which elements are added.

Front: queue-ൽ നിന്ന് elements delete ചെയ്യുന്ന അറ്റം. **Rear:** queue-ലേക്ക് elements ചേർക്കുന്ന അറ്റം.

Unlike a stack, where both operations happen at the top, a queue uses two ends. Adding moves the rear pointer to the new element; deleting moves the front pointer to the next element. Queues are usually drawn horizontally with front on the left.

മലയാളത്തിൽ

Stack-ൽ എല്ലാം top-ൽ മാത്രം. Queue-ൽ രണ്ട് അറ്റം: പുതിയ ആൾ rear-ൽ ചേരുന്നു, service കിട്ടിയ ആൾ front-ൽ നിന്ന് പോകുന്നു. ചേർക്കുമ്പോൾ rear മുന്നോട്ട്, delete ചെയ്യുമ്പോൾ front മുന്നോട്ട്.

Addition: 1. Create the new element. 2. If the queue is empty, make front and rear point to it. 3. Else add it after rear and move rear to it.

Deletion: 1. If the queue is empty, print "Queue Empty". 2. Else delete the front element: if it was the only element, make front and rear NULL; else move front to the next element.

EXAMPLE (THE UNIT'S FIGURES 5.1 TO 5.3)

Queue: a b c d e f (front = a, rear = f). Add g, then delete one element.

Operation	Queue (front ... rear)	front	rear
start	a b c d e f	a	f
add g	a b c d e f g	a	g
delete	b c d e f g (a removed)	b	g

Exam tip: Check Your Progress 1: "In a queue, addition takes place at the **rear** and deletion at the **front**." "A queue is also known as a **FIFO** (First In First Out) list." A stack is LIFO; do not mix them up.

Array implementation of a queue

Array implementation of a queue: storing the queue in a fixed-size array with two integer indices, front and rear, marking the first and last elements.

Array implementation of a queue: ഒരു fixed-size array-ൽ queue സൂക്ഷിച്ച്, ആദ്യത്തെയും അവസാനത്തെയും elements കാണിക്കാൻ front, rear എന്ന രണ്ട് integer indices ഉപയോഗിക്കുന്നത്.

We start with front = 0 and rear = -1 (empty). To add, check overflow (rear = size - 1), then increment rear and store. To delete, check empty (front > rear), then take element[front] and increment front. Both operations take O(1) time.

മലയാളത്തിൽ

ഒരു നിരയിലെ കണ്ടേറകൾ പോലെ array. rear എന്ന index പുതിയ ആളെ ഇരുത്തുന്ന കണ്ടേറ, front ആദ്യം എഴുന്നേൽക്കേണ്ട ആൾ. പ്രശ്നം: front-ലെ ആളുകൾ പോയാലും ആ കണ്ടേറകൾ വീണ്ടും ഉപയോഗിക്കുന്നില്ല; rear അവസാനം എത്തിയാൽ "overflow" എന്ന് പറയും, മുന്നിൽ കണ്ടേറ ഒഴിഞ്ഞു കിടന്നാലും.

Initial (empty): front = 0, rear = -1

Empty: front > rear

Full (overflow): rear = QUEUE_LENGTH - 1

Add: element[++rear] = x

Delete: x = element[front++]

Number of elements = rear - front + 1

Corrected Program 5.1 (with a fixed demo in main instead of a one-shot menu):

```

#include <stdio.h>
#define QUEUE_LENGTH 5

struct queue {
    int element[QUEUE_LENGTH];
    int front, rear;
};

struct queue q = { {0}, 0, -1 }; /* empty queue: front = 0, rear = -1 */

void add(int y)
{
    if (q.rear == QUEUE_LENGTH - 1)
        printf("Queue overflow\n");
    else
        q.element[++q.rear] = y;
}

int delete_element(void)
{
    if (q.front > q.rear) {
        printf("Queue empty\n");
        return -1;
    }
    return q.element[q.front++];
}

int main(void)
{
    add(10); add(20); add(30);
    printf("Deleted %d\n", delete_element());
    printf("Deleted %d\n", delete_element());
    add(40); add(50);
    printf("front = %d, rear = %d\n", q.front, q.rear);
    add(60);
    return 0;
}

```

Output:

```

Deleted 10
Deleted 20
front = 2, rear = 4
Queue overflow

```

EXAMPLE: TRACE OF THE PROGRAM (QUEUE_LENGTH = 5)

Operation	element[0 . . 4]	front	rear	Result
start	_____	0	-1	empty
add 10	10 _____	0	0	
add 20	10 20 ____	0	1	
add 30	10 20 30 __	0	2	
delete	_ 20 30 __	1	2	10
delete	__ 30 __	2	2	20
add 40	__ 30 40 _	2	3	
add 50	__ 30 40 50	2	4	
add 60	__ 30 40 50	2	4	overflow

Only 3 elements are stored, yet "Queue overflow" is printed because $\text{rear} = 4 = \text{QUEUE_LENGTH} - 1$. Slots 0 and 1 are wasted. This is exactly the problem the circular queue solves.

Exam tip: Program 5.1 in the unit does not compile and has logic bugs: `struct queue {...}` lacks its closing semicolon, several statements lack semicolons, `add(&q, x)` is called with two arguments but defined as `add(y)`, `delete()` uses an undeclared `x` and misspells `return`, and `if q.front > q.rear` needs brackets. Worse, `front` and `rear` are never initialised (both 0), so the first `add` goes to `element[1]` and an empty queue is reported as non-empty. The corrected version above initialises `front = 0`, `rear = -1`, checks overflow before incrementing `rear`, and uses `int main(void)`. (`delete` is also a C++ keyword, so we name the function `delete_element`.)

Linked list implementation of a queue

Node: a record with a data field and a pointer (Ptrnext) to the next node.

Linked queue: a queue stored as a singly linked list of nodes, with a front pointer to the first node (deletion end) and a rear pointer to the last node (insertion end).

Node: ഒരു data field-ഉം അടുത്ത node-ലേക്കുള്ള ഒരു pointer-ഉം (Ptrnext) ഉള്ള record.

Linked queue: nodes-ന്റെ singly linked list ആയി സൂക്ഷിക്കുന്ന queue; ആദ്യ node-ലേക്ക് front pointer (delete അറ്റം), അവസാന node-ലേക്ക് rear pointer (ചേർക്കുന്ന അറ്റം).

Each new node is created with `malloc` and linked after rear, so the queue grows as long as memory is available: there is no overflow at a fixed size. Deleting frees the front node and moves front to the next node. If the last node is deleted, both front and rear become NULL.

മലയാളത്തിൽ

Token system ഉള്ള ആശുപത്രി പോലെ: ഓരോ ആൾക്കും "എന്റെ പിറകിൽ ആർ" എന്ന് അറിയാം (next pointer). എത്ര പേർ വന്നാലും ചേർക്കാം, കസേരയുടെ എണ്ണം പ്രശ്നമല്ല. ഒരേ ഒരു node ബാക്കിയുള്ളപ്പോൾ delete ചെയ്താൽ front-ഉം rear-ഉം NULL ആക്കാൻ മറക്കരുത്.

Empty: `front == NULL`

Add: `new->next = NULL`; if empty, `front = rear = new`; else `rear->next = new`, `rear = new`

Delete: `temp = front`; `front = front->next`; if `front == NULL` then `rear = NULL`; `free(temp)`

```

#include <stdio.h>
#include <stdlib.h>

struct node {
    int value;
    struct node *next;
};

struct node *front = NULL, *rear = NULL;

void add(int value)
{
    struct node *new = malloc(sizeof(struct node));
    if (new == NULL) { printf("No memory\n"); return; }
    new->value = value;
    new->next = NULL;
    if (front == NULL)          /* queue was empty */
        front = rear = new;
    else {
        rear->next = new;
        rear = new;
    }
}

int delete_element(void)
{
    struct node *temp;
    int delvalue;
    if (front == NULL) {
        printf("Queue Empty\n");
        return -1;
    }
    temp = front;
    delvalue = temp->value;
    front = front->next;
    if (front == NULL)          /* it was the only node */
        rear = NULL;
    free(temp);
    return delvalue;
}

int main(void)
{
    add(5); add(8); add(3);
    printf("Deleted %d\n", delete_element());
    add(9);
    while (front != NULL)
        printf("Deleted %d\n", delete_element());
    delete_element();
}

```

```
    return 0;
}
```

Output:

```
Deleted 5
Deleted 8
Deleted 3
Deleted 9
Queue Empty
```

EXAMPLE: TRACE

Operation	List (front → ... → rear)	front	rear
start	(empty)	NULL	NULL
add 5	5 → NULL	5	5
add 8	5 → 8 → NULL	5	8
add 3	5 → 8 → 3 → NULL	5	3
delete (5)	8 → 3 → NULL	8	3
add 9	8 → 3 → 9 → NULL	8	9
delete ×3 (8, 3, 9)	(empty)	NULL	NULL

EXAMPLE 2 (CHECK YOUR PROGRESS 1, Q3)

Compare the array and linked list representations of a queue.

Point	Array	Linked list
Size	Fixed at compile time; overflow when full	Grows and shrinks at run time; limited only by memory
Extra memory	None per element	One pointer per node
Space reuse	Linear array wastes freed front slots	Freed nodes return to memory
Access	Any element by index	Only by following pointers
Add / delete time	$O(1)$	$O(1)$ (with both front and rear pointers)

Exam tip: The unit's Programs 5.2 and 5.3 have bugs: `sizeof(queue)` should be `sizeof(struct queue)` (the node structure), the extra `queueptr` variable is unnecessary, and in Program 5.3 the `else` after `printf("Queue Empty")` is missing, so on an empty queue it goes on to read `front->value` through a `NULL` pointer. It also never returns `delvalue`. The version above fixes all of these.

Multiple queues

Multiqueue: a data structure in which several queues are maintained together, for example in one array divided into segments, each with its own front and rear.

Multiqueue: പല queues ഒരുമിച്ച് സൂക്ഷിക്കുന്ന data structure; ഉദാഹരണത്തിന് ഒരു array-യെ ഭാഗങ്ങളായി തിരിച്ച്, ഓരോ ഭാഗത്തിനും സ്വന്തം front-ഉം rear-ഉം.

Many applications, such as process scheduling with several priority levels, need several queues. A one-dimensional array of m elements can hold k queues of n elements each: queue 1 uses positions $1 \dots n$, queue 2 uses $n + 1 \dots 2n$, and so on. Each queue i has its own `front[i]` and `rear[i]`.

മലയാളത്തിൽ

ഒരു bank-ൽ ഒരു hall-ൽ cash counter, loan counter, enquiry counter എന്നിങ്ങനെ പല വരികൾ ഉള്ളതുപോലെ. ഒരു വലിയ array-യെ k ഭാഗങ്ങളാക്കി ഓരോ ഭാഗവും ഓരോ queue. ഓരോ queue-നും സ്വന്തം $\text{front}[i]$, $\text{rear}[i]$; അതിർത്തി കടന്നാൽ ആ queue full.

k queues of n elements in one array (C indices $0 \dots kn - 1$):

$\text{bound}[i] = i \cdot n - 1$ (fixed, for $i = 0 \dots k$)

Initially $\text{front}[i] = \text{rear}[i] = \text{bound}[i]$

Queue i empty: $\text{front}[i] == \text{rear}[i]$

Queue i full: $\text{rear}[i] == \text{bound}[i + 1]$

Add: $\text{mqueue}[++\text{rear}[i]] = x$ Delete: $x = \text{mqueue}[++\text{front}[i]]$

```

#include <stdio.h>
#define K 3          /* number of queues */
#define N 4          /* size of each queue */

int mqueue[K * N];
int front[K], rear[K], bound[K + 1];

void init(void)
{
    int i;
    for (i = 0; i <= K; i++)
        bound[i] = i * N - 1;          /* fixed boundaries */
    for (i = 0; i < K; i++)
        front[i] = rear[i] = bound[i]; /* one before the first slot */
}

void addmq(int i, int x)          /* add x to queue i */
{
    if (rear[i] == bound[i + 1])
        printf("Queue %d is full\n", i);
    else
        mqueue[++rear[i]] = x;
}

int delmq(int i)                  /* delete from queue i */
{
    if (front[i] == rear[i]) {
        printf("Queue %d is empty\n", i);
        return -1;
    }
    return mqueue[++front[i]];
}

int main(void)
{
    init();
    addmq(0, 11); addmq(0, 12);
    addmq(1, 21);
    addmq(2, 31); addmq(2, 32); addmq(2, 33); addmq(2, 34);
    addmq(2, 35);
    printf("Deleted %d from queue 0\n", delmq(0));
    printf("Deleted %d from queue 2\n", delmq(2));
    printf("Deleted %d from queue 1\n", delmq(1));
    delmq(1);
    return 0;
}

```

Output:

```

Queue 2 is full
Deleted 11 from queue 0
Deleted 31 from queue 2
Deleted 21 from queue 1
Queue 1 is empty

```

EXAMPLE: TRACE (K = 3, N = 4, ARRAY INDICES 0 ... 11)

bound = -1, 3, 7, 11. Queue 0 uses 0-3, queue 1 uses 4-7, queue 2 uses 8-11.

Operation	front[0], rear[0]	front[1], rear[1]	front[2], rear[2]	Result
init	-1, -1	3, 3	7, 7	all empty
add 11, 12 to Q0	-1, 1	3, 3	7, 7	mq[0]=11, mq[1]=12
add 21 to Q1	-1, 1	3, 4	7, 7	mq[4]=21
add 31, 32, 33, 34 to Q2	-1, 1	3, 4	7, 11	mq[8..11]
add 35 to Q2	-1, 1	3, 4	7, 11	full (rear[2] = bound[3] = 11)
delete Q0	0, 1	3, 4	7, 11	11
delete Q2	0, 1	3, 4	8, 11	31
delete Q1	0, 1	4, 4	8, 11	21
delete Q1	0, 1	4, 4	8, 11	empty

Exam tip: Check Your Progress 2, Q2: "It is not possible to implement multiple queues in an array" is **False**. The unit's Programs 5.4 and 5.5 are wrong: 5.4 increments rear[i] twice (+rear[i] and then rear[i] = rear[i]+1) and compares with front[i+1], which moves as queue i+1 is used; 5.5 lacks an else and decrements front[i], which walks backwards into the previous queue. Use a fixed boundary and increment front on delete, as above.

Circular queue: idea and algorithms

Circular queue: a queue in which the last position is treated as connected back to the first, so that after index $n - 1$ the next position is index 0, allowing freed front positions to be reused.

Circular queue: അവസാന position ആദ്യ position-ലേക്ക് ചേർന്നിരിക്കുന്നതായി കണക്കാക്കുന്ന queue; index $n - 1$ കഴിഞ്ഞാൽ അടുത്തത് index 0. അതിനാൽ front-ൽ ഒഴിഞ്ഞ positions വീണ്ടും ഉപയോഗിക്കാം.

A linear array queue wastes space: once rear reaches the last index it reports overflow even if elements have been deleted from the front. In a circular queue, when rear is at $n - 1$ the next element goes to index 0 if that slot is free. Following the unit, front points one position before the first element (anticlockwise), and the queue is empty when front and rear are equal.

മലയാളത്തിൽ

Giant wheel (ജയന്റ് വീൽ) പോലെ ചിന്തിക്കുക: ഏറ്റവും അവസാനത്തെ cabin കഴിഞ്ഞാൽ വീണ്ടും ആദ്യത്തേത് വരുന്നു. Queue-ന്റെ അവസാന index കഴിഞ്ഞാൽ rear വീണ്ടും 0-ലേക്ക് പോകും, അതുകൊണ്ട് front-ൽ നിന്ന് ആളുകൾ പോയ ഒഴിഞ്ഞ സ്ഥലം വീണ്ടും കിട്ടും. ഇത് ചെയ്യാൻ % (modulus) operator ആണ് എളുപ്പം.

Array of size MAX, indices 0 ... MAX - 1

Next position after i: $(i + 1) \% \text{MAX}$

Initially front = rear = 0

Empty: front == rear

Full: $(\text{rear} + 1) \% \text{MAX} == \text{front}$ (one slot is kept empty, so at most MAX - 1 elements)

Add: rear = $(\text{rear} + 1) \% \text{MAX}$; queue[rear] = x

Delete: front = $(\text{front} + 1) \% \text{MAX}$; x = queue[front]

Number of elements = $(\text{rear} - \text{front} + \text{MAX}) \% \text{MAX}$

EXAMPLE (THE UNIT'S FIGURE 5.7)

The figure shows Front = 0 and Rear = 4 with 10, 50, 15, 20 at positions 1, 2, 3, 4. How many elements are there, and which is deleted first?

Count = $(4 - 0 + n) \% n = 4$. Front points one position before the first element, so the first element is at $(0 + 1) = \text{position } 1$, i.e. 10 is deleted first, and the new front becomes 1.

EXAMPLE 2

MAX = 8, front = 6, rear = 2. How many elements, and where does the next element go?

Count = $(2 - 6 + 8) \% 8 = 4$ (positions 7, 0, 1, 2). Next add goes to $(2 + 1) \% 8 = 3$. The queue is full when $(\text{rear} + 1) \% 8 = 6$, i.e. rear = 5, holding 7 elements.

Exam tip: The unit says that when the circular queue is created "both front and rear point to index 1"; this is a slip. With the unit's own convention (front one position before the first element, Figure 5.7 with Front = 0), they start at index 0, and the answer key (Check Your Progress 2, Q3) also says the index starts at 0. The unit's add algorithm detects overflow by checking for a "not NULL value" in the slot; that fails when 0 is a stored value, so the standard test $(\text{rear} + 1) \% \text{MAX} == \text{front}$ is used instead.

Array implementation of a circular queue

Array circular queue: a circular queue stored in an array of size MAX in which front and rear advance using $(\text{index} + 1) \% \text{MAX}$, so they wrap from MAX - 1 to 0.

Array circular queue: MAX size ഉള്ള array-ൽ സൂക്ഷിക്കുന്ന circular queue; front-ഉം rear-ഉം $(\text{index} + 1) \% \text{MAX}$ ഉപയോഗിച്ച് മുന്നോട്ട് നീങ്ങുന്നു, അതിനാൽ MAX - 1 കഴിഞ്ഞാൽ 0-ലേക്ക് തിരിയുന്നു.

The unit's two algorithms (move rear, wrapping to 0 at the last position; move front, wrapping to 0 at the last position) are written compactly with the % operator. We check fullness before moving rear, so a failed add leaves the queue unchanged.

മലയാളത്തിൽ

```
if (rear == max-1) rear = 0; else rear = rear + 1; എന്നത് തന്നെയാണ്
rear = (rear + 1) % MAX; . ചേർക്കും മുമ്പ് full ആണോ എന്ന് നോക്കണം;
അല്ലെങ്കിൽ rear തെറ്റായ സ്ഥലത്ത് എത്തി പഴയ data മാജ്ക്കം.
```

```

#include <stdio.h>
#define MAX 5

int queue[MAX];
int front = 0, rear = 0; /* empty when front == rear */

void add(int y)
{
    int next = (rear + 1) % MAX;
    if (next == front)
        printf("Queue Overflow\n");
    else {
        rear = next;
        queue[rear] = y;
    }
}

int deleteelement(void)
{
    if (front == rear) {
        printf("Error - Queue empty\n");
        return -1;
    }
    front = (front + 1) % MAX;
    return queue[front];
}

int main(void)
{
    add(10); add(20); add(30); add(40);
    add(50);
    printf("Deleted %d\n", deleteelement());
    printf("Deleted %d\n", deleteelement());
    add(50); add(60);
    printf("front = %d, rear = %d\n", front, rear);
    add(70);
    while (front != rear)
        printf("Deleted %d\n", deleteelement());
    deleteelement();
    return 0;
}

```

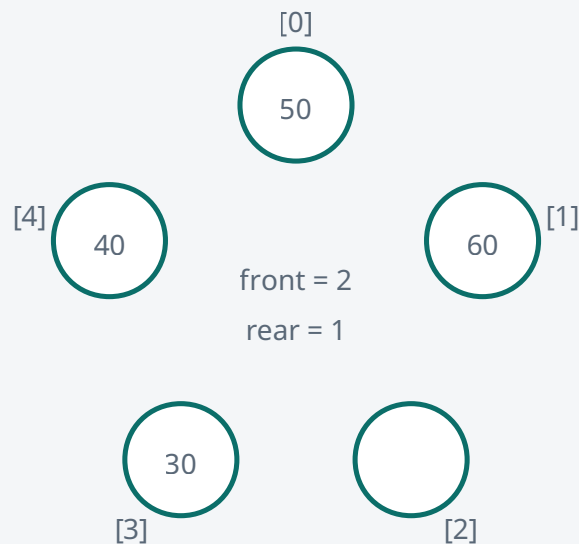
Output:

```
Queue Overflow  
Deleted 10  
Deleted 20  
front = 2, rear = 1  
Queue Overflow  
Deleted 30  
Deleted 40  
Deleted 50  
Deleted 60  
Error - Queue empty
```

EXAMPLE: TRACE (MAX = 5, SO AT MOST 4 ELEMENTS)

Operation	queue[0 . . 4] (live elements)	front	rear	Result
start	_____	0	0	empty
add 10	_ 10 _ _ _	0	1	
add 20	_ 10 20 _ _	0	2	
add 30	_ 10 20 30 _	0	3	
add 40	_ 10 20 30 40	0	4	
add 50	unchanged	0	4	overflow: $(4+1)\%5 = 0 = \text{front}$
delete	_ _ 20 30 40	1	4	10
delete	_ _ _ 30 40	2	4	20
add 50	50 _ _ 30 40	2	0	rear wraps: $(4+1)\%5 = 0$
add 60	50 60 _ 30 40	2	1	
add 70	unchanged	2	1	overflow: $(1+1)\%5 = 2 = \text{front}$
delete	50 60 _ _ 40	3	1	30
delete	50 60 _ _ _	4	1	40
delete	_ 60 _ _ _	0	1	50 (front wraps)
delete	_____	1	1	60, now empty
delete	_____	1	1	queue empty

Compare with the linear queue in topic 3: here 50 and 60 reuse slots 0 and 1 that were freed at the front.



State after "add 60": slot 2 (front) is the kept-empty slot; the queue in order is 30, 40, 50, 60, so it is full.

Exam tip: Program 5.6 in the unit uses `void main()` (use `int main(void)`) and has real logic bugs. Its empty test `front == NULL` means `front == 0`, which is true at the start and stays true, so no element is ever deleted. It tests fullness with `queue[front] != NULL`, which fails if 0 is stored. And it moves rear before the overflow check: with `max = 10`, the 11th add sets rear to 0 and prints "Queue Overflow" but leaves rear there, so the 12th add silently overwrites `queue[1]`. The corrected program above checks `(rear + 1) % MAX == front` first.

Linked list implementation of a circular queue

Circular linked queue: a queue stored as a linked list in which the last node's next pointer points back to the first node, so a single rear pointer gives both ends (`front = rear->next`).

Circular linked queue: അവസാന node-ന്റെ next pointer ആദ്യ node-ലേക്ക് തിരിച്ചു ചൂണ്ടുന്ന linked list ആയി സൂക്ഷിക്കുന്ന queue; അതിനാൽ ഒരു rear pointer കൊണ്ട് രണ്ട് അറ്റവും കിട്ടും (`front = rear->next`).

The linked version has no fixed capacity and uses space efficiently, at the cost of one pointer per node. Because the list is a ring, we keep only rear; the front node is always `rear->next`. A single node points to itself.

മലയാളത്തിൽ

ഒരു വട്ടത്തിൽ കൈ കോർത്ത് നിൽക്കുന്ന കുട്ടികളെ ഓർക്കുക: അവസാനത്തെ കുട്ടി ആദ്യത്തെ കുട്ടിയുടെ കൈ പിടിച്ചിരിക്കുന്നു. അതുകൊണ്ട് അവസാനത്തെ ആളെ (rear) അറിഞ്ഞാൽ മതി, അടുത്തയാൾ front ആണ്.

Empty: rear == NULL

Add to empty: new->next = new; rear = new

Add otherwise: new->next = rear->next; rear->next = new; rear = new

Delete: front = rear->next; if front == rear then rear = NULL else rear->next = front->next; free(front)

```

#include <stdio.h>
#include <stdlib.h>

struct cq {
    int value;
    struct cq *next;
};

struct cq *rear = NULL;    /* rear->next is the front */

void add(int value)
{
    struct cq *new = malloc(sizeof(struct cq));
    if (new == NULL) { printf("No memory\n"); return; }
    new->value = value;
    if (rear == NULL)
        new->next = new;        /* single node points to itself */
    else {
        new->next = rear->next; /* new node points to front */
        rear->next = new;
    }
    rear = new;
}

int delete_element(void)
{
    struct cq *front;
    int delvalue;
    if (rear == NULL) {
        printf("Queue is empty\n");
        return -1;
    }
    front = rear->next;
    delvalue = front->value;
    if (front == rear)        /* only one node */
        rear = NULL;
    else
        rear->next = front->next;
    free(front);
    return delvalue;
}

int main(void)
{
    add(1); add(2); add(3);
    printf("Front = %d, Rear = %d\n", rear->next->value, rear->value);
    printf("Deleted %d\n", delete_element());
    add(4);
    while (rear != NULL)
        printf("Deleted %d\n", delete_element());
    delete_element();
}

```

```

    return 0;
}

```

Output:

```

Front = 1, Rear = 3
Deleted 1
Deleted 2
Deleted 3
Deleted 4
Queue is empty

```

EXAMPLE: TRACE

Operation	Ring (front → ... → rear → back to front)	front (rear->next)	rear
start	(empty)	–	NULL
add 1	1 → 1	1	1
add 2	1 → 2 → 1	1	2
add 3	1 → 2 → 3 → 1	1	3
delete (1)	2 → 3 → 2	2	3
add 4	2 → 3 → 4 → 2	2	4
delete ×3 (2, 3, 4)	(empty)	–	NULL

Exam tip: Program 5.7 in the unit is not actually circular: rear->next is never pointed back to the front, so it is just Program 5.2 again. It also declares `int *next` instead of `struct cq *next`, misses the semicolon after the typedef, assigns to the type name (`cqptr = front = ...`), uses the undeclared `queueptr`, and in delete reads `front->value` inside the "Queue is empty" branch (NULL dereference). The program above links the last node back to the first.

Deque (double ended queue)

Deque (double ended queue): an abstract data type similar to a queue in which elements can be added and deleted at both ends, front and rear.

Deque (double ended queue): queue പോലെയുള്ള ഒരു abstract data type; ഇതിൽ front, rear എന്ന രണ്ട് അറ്റങ്ങളിലും elements ചേർക്കാനും delete ചെയ്യാനും കഴിയും.

A dequeue supports four operations: add at front, add at rear, delete from front, delete from rear. If we use only add-at-rear and delete-from-front it behaves as a queue; if we use only one end it behaves as a stack. Two common restricted forms are the input-restricted dequeue (add at one end only, delete at both) and the output-restricted dequeue (delete at one end only, add at both).

മലയാളത്തിൽ

KSRTC ബസിന് മുന്നിലും പിന്നിലും വാതിൽ ഉള്ളതുപോലെ: രണ്ട് വാതിലിലൂടെയും കയറാം, ഇറങ്ങാം. ഒരു അറ്റം മാത്രം ഉപയോഗിച്ചാൽ അത് stack പോലെ, rear-ൽ കയറി front-ൽ ഇറങ്ങിയാൽ സാധാരണ queue പോലെ.

Operations: add_front(x), add_rear(x), delete_front(), delete_rear()

Input-restricted dequeue: insertion at one end only, deletion at both ends

Output-restricted dequeue: deletion at one end only, insertion at both ends

EXAMPLE

Empty dequeue. add_rear(10), add_rear(20), add_front(5), delete_rear, add_front(1), delete_front. Show the contents after each step.

Operation	Dequeue (front ... rear)	Returned
add_rear(10)	10	
add_rear(20)	10 20	
add_front(5)	5 10 20	
delete_rear	5 10	20
add_front(1)	1 5 10	
delete_front	5 10	1

Exam tip: Check Your Progress 2, Q1: "**Dequeue** allows elements to be added and deleted at the front as well as at the rear." Many books spell the structure "deque" (pronounced "deck") to avoid confusion with the delete operation "dequeue"; IGNOU uses "Dequeue", so write that in the IGNOU exam.

Array implementation of a dequeue

Array dequeue: a dequeue stored in an array with front and rear indices, where `add_front` moves front back by one and `add_rear` moves rear forward by one; making the array circular lets both ends wrap around.

Array dequeue: front, rear indices ഉള്ള array-ൽ സൂക്ഷിക്കുന്ന dequeue; `add_front` front-നെ ഒന്ന് പിന്നോട്ടും `add_rear` rear-നെ ഒന്ന് മുന്നോട്ടും നീക്കുന്നു. Array circular ആക്കിയാൽ രണ്ട് അറ്റവും wrap ചെയ്യും.

The unit notes that a linear array dequeue suffers the same wasted-space problem as a linear queue; in addition, once front is at index 0 nothing can be added at the front. Treating the array as circular fixes both: moving front back from 0 goes to $\text{MAX} - 1$. We use `front = rear = -1` for empty, and the dequeue is full when $(\text{rear} + 1) \% \text{MAX} == \text{front}$, so all MAX slots can be used.

മലയാളത്തിൽ

Linear array-ൽ front 0-ൽ എത്തിയാൽ പിന്നെ മുന്നിൽ ചേർക്കാൻ സാധ്യമില്ല.

Circular ആക്കിയാൽ 0-ന് "മുന്നിൽ" $\text{MAX} - 1$ ആണ്. front പിന്നോട്ട് നീക്കുമ്പോൾ $(\text{front} - 1 + \text{MAX}) \% \text{MAX}$ എഴുതണം; വെറും `front - 1` എഴുതിയാൽ -1 ആയി array-ക്ക് പുറത്തു പോകും.

Empty: `front == -1` (then `front = rear = 0` on the first add)

Full: `(rear + 1) \% MAX == front`

`add_front`: `front = (front - 1 + MAX) \% MAX`; `dq[front] = x`

`add_rear`: `rear = (rear + 1) \% MAX`; `dq[rear] = x`

`delete_front`: `x = dq[front]`; `front = (front + 1) \% MAX` (if `front == rear`, set both to -1)

`delete_rear`: `x = dq[rear]`; `rear = (rear - 1 + MAX) \% MAX` (if `front == rear`, set both to -1)

```

#include <stdio.h>
#define QUEUE_LENGTH 5

int dq[QUEUE_LENGTH];
int front = -1, rear = -1;          /* empty deque */

int is_full(void) { return (rear + 1) % QUEUE_LENGTH == front; }
int is_empty(void) { return front == -1; }

void add_front(int y)
{
    if (is_full()) { printf("Deque full\n"); return; }
    if (is_empty())
        front = rear = 0;
    else
        front = (front - 1 + QUEUE_LENGTH) % QUEUE_LENGTH;
    dq[front] = y;
}

void add_rear(int y)
{
    if (is_full()) { printf("Deque full\n"); return; }
    if (is_empty())
        front = rear = 0;
    else
        rear = (rear + 1) % QUEUE_LENGTH;
    dq[rear] = y;
}

int delete_front(void)
{
    int x;
    if (is_empty()) { printf("Deque empty\n"); return -1; }
    x = dq[front];
    if (front == rear)
        front = rear = -1;
    else
        front = (front + 1) % QUEUE_LENGTH;
    return x;
}

int delete_rear(void)
{
    int x;
    if (is_empty()) { printf("Deque empty\n"); return -1; }
    x = dq[rear];
    if (front == rear)
        front = rear = -1;
    else
        rear = (rear - 1 + QUEUE_LENGTH) % QUEUE_LENGTH;
    return x;
}

```

```

}

int main(void)
{
    add_rear(10); add_rear(20);
    add_front(5);
    printf("front = %d, rear = %d\n", front, rear);
    add_front(2); add_rear(30);
    add_rear(40);
    printf("Deleted from rear: %d\n", delete_rear());
    printf("Deleted from front: %d\n", delete_front());
    printf("front = %d, rear = %d\n", front, rear);
    return 0;
}

```

Output:

```

front = 4, rear = 1
Deque full
Deleted from rear: 30
Deleted from front: 2
front = 4, rear = 1

```

EXAMPLE: TRACE (QUEUE_LENGTH = 5)

Operation	dq[0..4]	front	rear	Dequeue (front ... rear)	Result
start	_____	-1	-1	(empty)	
add_rear 10	10_____	0	0	10	
add_rear 20	10 20____	0	1	10 20	
add_front 5	10 20__5	4	1	5 10 20	front wraps ($0 - 1 + 5$) $\% 5 = 4$
add_front 2	10 20_2 5	3	1	2 5 10 20	
add_rear 30	10 20 30 2 5	3	2	2 5 10 20 30	now full
add_rear 40	unchanged	3	2	2 5 10 20 30	full: $(2+1)\%5 = 3 =$ front
delete_rear	10 20_2 5	3	1	2 5 10 20	30
delete_front	10 20__5	4	1	5 10 20	2

Exam tip: Program 5.8 in the unit has many bugs. `#define QUEUE_LENGTH 10;` (with the semicolon) breaks `int dq[QUEUE_LENGTH];`. On an empty dequeue (`front = -1`), `add_front` sets `front` to `-2` and writes `dq[-2]`, outside the array. `delete_front` returns before updating `front`, so `front` never moves. `add_rear` tests `front` instead of `rear` against `QUEUE_LENGTH - 1` and never sets `front` on the first insertion. `delete_rear` decrements `rear` first and then returns `dq[rear]`, i.e. the wrong element. `=` must be written `==`. The circular version above avoids all of these.

Linked list implementation of a dequeue

Doubly linked list: a linked list whose nodes have two pointers, `left` (to the previous node) and `right` (to the next node), so it can be traversed in both directions.

Linked dequeue: a dequeue stored as a doubly linked list with a head pointer (`front`) and a tail pointer (`rear`), giving $O(1)$ addition and deletion at both ends.

Doubly linked list: ഓരോ node-നും `left` (മുമ്പത്തെ node-ലേക്ക്), `right` (അടുത്ത node-ലേക്ക്) എന്ന രണ്ട് pointers ഉള്ള linked list; അതിനാൽ രണ്ട് ദിശയിലും traverse ചെയ്യാം.

Linked dequeue: `head` (`front`), `tail` (`rear`) pointers ഉള്ള doubly linked list ആയി സൂക്ഷിക്കുന്ന dequeue; രണ്ട് അറ്റത്തും $O(1)$ സമയത്തിൽ ചേർക്കലും `delete` ചെയ്യലും.

Deleting at the rear of a singly linked list needs the node before the tail, which takes a full traversal. With a left pointer in each node, the tail's previous node is just `tail->left`, so all four operations are $O(1)$. That is why dequeues are implemented with doubly linked lists.

മലയാളത്തിൽ

ട്രെയിനിലെ compartments പോലെ: ഓരോന്നും മുന്നിലുള്ളതുമായും പിന്നിലുള്ളതുമായും ചേർന്നിരിക്കുന്നു. അതുകൊണ്ട് എഞ്ചിൻ ഭാഗത്തുനിന്നോ അവസാന ഭാഗത്തുനിന്നോ ഒരു compartment എളുപ്പം ചേർക്കാനും മാറ്റാനും കഴിയും. Singly linked list-ൽ അവസാനത്തേത് മാറ്റാൻ മുഴുവൻ നടന്നു പോകണം.

Node: info, left, right

eq_front: `p->right = head; head->left = p; head = p`

eq_back: `p->left = tail; tail->right = p; tail = p`

dq_front: `head = head->right; if head == NULL then tail = NULL else head->left = NULL`

dq_back: `tail = tail->left; if tail == NULL then head = NULL else tail->right = NULL`

```

#include <stdio.h>
#include <stdlib.h>

struct dq {
    int info;
    struct dq *left, *right;
};

struct dq *head = NULL, *tail = NULL;

struct dq *getnode(int x)
{
    struct dq *p = malloc(sizeof(struct dq));
    if (p == NULL) { printf("No memory\n"); exit(1); }
    p->info = x;
    p->left = p->right = NULL;
    return p;
}

void eq_front(int x)          /* add at front */
{
    struct dq *p = getnode(x);
    if (head == NULL)
        head = tail = p;
    else {
        p->right = head;
        head->left = p;
        head = p;
    }
}

void eq_back(int x)          /* add at rear */
{
    struct dq *p = getnode(x);
    if (head == NULL)
        head = tail = p;
    else {
        p->left = tail;
        tail->right = p;
        tail = p;
    }
}

int dq_front(void)           /* delete at front */
{
    struct dq *tp = head;
    int x;
    if (head == NULL) { printf("Deque empty\n"); return -1; }
    x = tp->info;
    head = head->right;
    if (head == NULL) tail = NULL;
}

```

```

        else head->left = NULL;
        free(tp);
        return x;
    }

int dq_back(void)                /* delete at rear */
{
    struct dq *tp = tail;
    int x;
    if (tail == NULL) { printf("Deque empty\n"); return -1; }
    x = tp->info;
    tail = tail->left;
    if (tail == NULL) head = NULL;
    else tail->right = NULL;
    free(tp);
    return x;
}

void show(void)
{
    struct dq *p;
    printf("Deque:");
    for (p = head; p != NULL; p = p->right)
        printf(" %d", p->info);
    printf("\n");
}

int main(void)
{
    eq_back(10); eq_back(20); eq_front(5);
    show();
    printf("Deleted at back: %d\n", dq_back());
    printf("Deleted at front: %d\n", dq_front());
    show();
    dq_front();
    dq_back();
    return 0;
}

```

Output:

```

Deque: 5 10 20
Deleted at back: 20
Deleted at front: 5
Deque: 10
Deque empty

```

EXAMPLE: TRACE

Operation	List (NULL $\leftarrow$ head $\rightleftarrows$... $\rightleftarrows$ tail $\rightarrow$ NULL)	head	tail	Returned
start	(empty)	NULL	NULL	
eq_back 10	10	10	10	
eq_back 20	10 $\rightleftarrows$ 20	10	20	
eq_front 5	5 $\rightleftarrows$ 10 $\rightleftarrows$ 20	5	20	
dq_back	5 $\rightleftarrows$ 10	5	10	20
dq_front	10	10	10	5
dq_front	(empty)	NULL	NULL	10
dq_back	(empty)	NULL	NULL	"Deque empty"

Exam tip: Program 5.9 in the unit is pseudo-code, not compilable C: left/right are declared `int *` instead of `struct dq *`, `dcons` is never defined, the parameter type `dq` is a struct name used without `struct`, `create_list` makes unrelated nodes (and writes `p_right`), if `dq` is not empty is not C, and `dq_back` tests `q != NULL` instead of emptiness. The program above is a working version of the same idea (the menu is replaced by a fixed demo).

Quick recap

- A queue is a FIFO linear list: add at the rear, delete from the front.
- Queue ADT: `createQueue`, `isEmpty`, `addQueue`, `deleteQueue`, `next`; deleting from an empty queue is an error (underflow).
- Array queue: `front = 0`, `rear = -1` initially; empty when `front > rear`; overflow when `rear = size - 1`.
- A linear array queue wastes the slots freed at the front; it can report overflow while partly empty.
- Linked queue: front and rear pointers; no fixed size; one extra pointer per node; set `rear = NULL` when the last node is deleted.
- Multiqueue: `k` queues of `n` elements in one array, each with its own `front[i]` and `rear[i]` and fixed boundaries.
- Circular queue: `next index = (i + 1) % MAX`; front points one position before the first element; empty when `front == rear`; full when `(rear + 1) % MAX == front`.
- A circular array queue of size `MAX` holds at most `MAX - 1` elements with this convention; `count = (rear - front + MAX) % MAX`.
- Circular linked queue: last node points back to the first; keep only rear, `front = rear->next`.

- Dequeue: add and delete at both ends; input-restricted and output-restricted are its two restricted forms.
- An array dequeue works best as a circular array: front moves back with $(\text{front} - 1 + \text{MAX}) \% \text{MAX}$.
- A linked dequeue uses a doubly linked list (left and right pointers), giving $O(1)$ operations at both ends.
- All add/delete operations in every version here are $O(1)$.
- The unit's C programs contain many errors (void main, missing semicolons, missing else, NULL checks on indices); know the corrected logic.

Check yourself

Q. 1. In a queue, addition takes place at ____ and deletion at _____. A queue is also called a ____ list.

Rear; front. FIFO (First In First Out) list.

Q. 2. Compare the array and linked list representations of a queue.

Array: fixed size decided in advance, so it can overflow; no pointer overhead; a linear array wastes slots freed at the front (fixed by making it circular). Linked list: grows and shrinks at run time, limited only by available memory; needs one extra pointer per node; freed nodes are returned to memory. Both give $O(1)$ add and delete when front and rear are kept.

Q. 3. A linear array queue of size 6 starts empty (front = 0, rear = -1). Perform: add 4, add 7, add 1, delete, add 9, delete. Give front, rear and the queue.

add 4: rear 0. add 7: rear 1. add 1: rear 2. delete → 4, front 1. add 9: rear 3. delete → 7, front 2.
Final: front = 2, rear = 3, queue = 1 9.

Q. 4. Why is a circular queue better than a linear queue? Give the full and empty conditions.

In a linear array queue, positions freed by deletions at the front are never reused, so the queue can report overflow while it has free slots. In a circular queue, rear moves from $\text{MAX} - 1$ back to 0, reusing those slots. With front = rear = 0 initially: empty when front == rear; full when $(\text{rear} + 1) \% \text{MAX} == \text{front}$.

Q. 5. A circular queue has MAX = 6, front = 0, rear = 0. Perform: add A, B, C, D, E; add F; delete; delete; add F; add G. Give front, rear and the result of each step.

add A–E: rear = 1, 2, 3, 4, 5 (A at 1 ... E at 5).

add F: $(5 + 1) \% 6 = 0 = \text{front} \rightarrow \text{overflow}$.

delete: front = 1, returns A. delete: front = 2, returns B.

add F: rear = $(5 + 1) \% 6 = 0$, queue[0] = F.

add G: rear = 1, queue[1] = G.

Final: front = 2, rear = 1; queue in order: C D E F G (5 elements = MAX – 1, so it is full).

Q. 6. A circular queue has MAX = 10, front = 7, rear = 3. How many elements does it hold?

$(\text{rear} - \text{front} + \text{MAX}) \% \text{MAX} = (3 - 7 + 10) \% 10 = 6$. They are at positions 8, 9, 0, 1, 2, 3.

Q. 7. ____ allows elements to be added and deleted at the front as well as at the rear. Name its two restricted forms.

Deque (double ended queue). Input-restricted deque (insert at one end, delete at both) and output-restricted deque (delete at one end, insert at both).

Q. 8. True or False: it is not possible to implement multiple queues in an array. The index of a circular queue starts at ____.

False: one array can be split into segments, each queue having its own front[i] and rear[i]. The index starts at 0.

Q. 9. A circular array deque of size 4 is empty (front = rear = -1). Perform: add_front 1, add_front 2, add_rear 3, delete_front, add_rear 4, add_rear 5. Give front, rear and the contents.

add_front 1: empty, so front = rear = 0, dq[0] = 1.

add_front 2: front = $(0 - 1 + 4) \% 4 = 3$, dq[3] = 2. Order: 2 1.

add_rear 3: rear = 1, dq[1] = 3. Order: 2 1 3.

delete_front: returns 2, front = $(3 + 1) \% 4 = 0$. Order: 1 3.

add_rear 4: rear = 2. Order: 1 3 4.

add_rear 5: rear = 3. Order: 1 3 4 5. Now $(3 + 1) \% 4 = 0 = \text{front}$, so it is full.

Final: front = 0, rear = 3.

Q. 10. Why is a dequeue implemented with a doubly linked list rather than a singly linked list?

Deleting at the rear needs the node before the tail. In a singly linked list that requires walking from the head, $O(n)$. In a doubly linked list it is `tail->left`, so delete at rear, like the other three operations, is $O(1)$.

UNIT 6

Trees

Data-യെ parent-child ബന്ധത്തിൽ, ഒരു hierarchy ആയി ക്രമീകരിക്കുന്ന non-linear data structure ആയ tree ആണ് ഈ unit-ൽ. General tree, binary tree എന്നിവയുടെ നിർവ്വചനം, properties, implementation, preorder/inorder/postorder traversal (recursive-ഉം stack ഉപയോഗിച്ചും), threaded binary tree, BST, expression tree പോലുള്ള applications എന്നിവ പഠിക്കുന്നു.

Tree as an abstract data type

Tree: definition and terminology

Full trees, complete trees and counting properties

Implementation of a general tree

Tree traversals: preorder, postorder, inorder, level order

Binary trees and their properties

Full and complete binary trees

Linked implementation and creation of a binary tree

Array representation of a binary tree

Binary tree traversals: recursive implementation

Non-recursive traversal using a stack

Threaded binary trees

Applications: BST, expression trees, game trees

Tree as an abstract data type

Abstract data type (ADT): a set of data values and associated operations that are precisely specified independent of any particular implementation.

Binary tree (as an ADT): either empty (nil), or fork(e, T, T'): an element e together with two sub-trees T and T' that are themselves binary trees.

Abstract data type (ADT): ഏതെങ്കിലും ഒരു implementation-നെ ആശ്രയിക്കാതെ കൃത്യമായി നിർവ്വചിച്ച data values-ഉം അവയുടെ operations-ഉം ചേർന്ന ഒരു set.

Binary tree (ADT ആയി): ഒന്നുകിൽ empty (nil), അല്ലെങ്കിൽ fork(e, T, T'): ഒരു element e -ഉം, സ്വയം binary tree-കളായ T, T' എന്ന രണ്ട് sub-tree-കളും.

Operating systems store files in a **hierarchy** of directories; a family tree goes grandparent $\rightarrow$ parent $\rightarrow$ you. A **tree** is the data structure for such parent-child relationships. The unit first describes a tree as an ADT: we say what the values and operations are, with mathematical precision, and not how they are coded. Because the definition is exact, we can reason about the operations before writing any program.

മലയാളത്തിൽ

നമ്മുടെ തറവാട്ടിലെ കുടുംബവൃക്ഷം ഓർക്കൂ: അപ്പുപ്പൻ മുകളിൽ, താഴെ മക്കൾ, അതിനു താഴെ പേരക്കുട്ടികൾ. ഇങ്ങനെ parent-child ബന്ധം സൂക്ഷിക്കാനാണ് tree. ADT എന്നാൽ "എന്തൊക്കെ values, എന്തൊക്കെ operations" എന്നു മാത്രം പറയുന്ന നിർവ്വചനം; C-യിലോ Python-ലോ എങ്ങനെ എഴുതും എന്നത് അവിടെ പ്രശ്നമല്ല. ഇവിടെ fork എന്ന operation ഒരു element-ഉം രണ്ട് sub-tree-കളും ചേർത്ത് പുതിയ tree ഉണ്ടാക്കുന്നു.

```
type Tree = nil | fork(Element, Tree, Tree)
null : Tree → Boolean  leaf : Tree → Boolean  fork : (Element, Tree, Tree) → Tree
left : Tree → Tree  right : Tree → Tree  contents : Tree → Element
height(nil) = 0; height(fork(e, T, T')) = 1 + max(height(T), height(T'))
weight(nil) = 0; weight(fork(e, T, T')) = 1 + weight(T) + weight(T')
```

Rule	Meaning
$\text{null}(\text{nil}) = \text{true}; \text{null}(\text{fork}(e, T, T')) = \text{false}$	nil is the empty tree
$\text{leaf}(\text{fork}(e, \text{nil}, \text{nil})) = \text{true}; \text{leaf}(\text{fork}(e, T, T')) = \text{false}$ if T or T' is not null; $\text{leaf}(\text{nil}) = \text{error}$	a leaf has two empty sub-trees
$\text{left}(\text{fork}(e, T, T')) = T; \text{right}(\text{fork}(e, T, T')) = T'$	left and right of a tree are again trees
$\text{contents}(\text{fork}(e, T, T')) = e$	the element stored at the root
$\text{left}(\text{nil}), \text{right}(\text{nil}), \text{contents}(\text{nil}) = \text{error}$	an empty tree has no parts

EXAMPLE

Write the tree of Figure 6.12 (root *, left child + with children 4 and 5, right child 3) using fork, and find its height and weight.

$T = \text{fork}(*, \text{fork}(+, \text{fork}(4, \text{nil}, \text{nil}), \text{fork}(5, \text{nil}, \text{nil})), \text{fork}(3, \text{nil}, \text{nil}))$.

A single leaf: $\text{height}(\text{fork}(4, \text{nil}, \text{nil})) = 1 + \max(0, 0) = 1$. So $\text{height}(+ \text{ sub-tree}) = 1 + \max(1, 1) = 2$ and $\text{height}(T) = 1 + \max(2, 1) = 3$.

$\text{weight}(\text{leaf}) = 1 + 0 + 0 = 1$; $\text{weight}(+ \text{ sub-tree}) = 1 + 1 + 1 = 3$; $\text{weight}(T) = 1 + 3 + 1 = 5$. The weight is simply the number of nodes.

Exam tip: In this ADT a tree with a single leaf has height 1 (not 0), because $\text{height}(\text{nil}) = 0$. Keep this convention for IGNOU answers: height = number of levels.

Tree: definition and terminology

Tree: a connected, acyclic graph; any node can be reached from any other node by exactly one path.

Rooted tree: a finite set of one or more nodes with one designated node r , the root; the remaining nodes are partitioned into disjoint subsets $T_1, \dots, T_k$, each a tree whose root is a child of r .

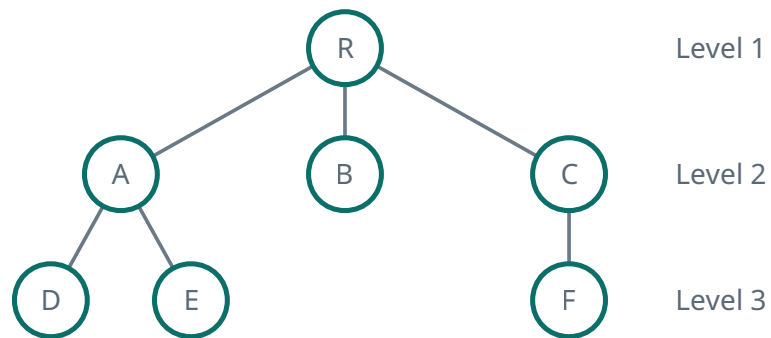
Tree: connected ആയതും cycle ഇല്ലാത്തതുമായ (acyclic) graph; ഏത് node-ൽ നിന്നും മറ്റേ node-ലേക്കും കൃത്യം ഒരു path മാത്രം.

Rooted tree: root എന്ന ഒരു പ്രത്യേക node r ഉള്ള, ഒന്നോ അതിലധികമോ node-കളുടെ finite set; ബാക്കി node-കൾ $T_1, \dots, T_k$ എന്ന disjoint subset-കളായി പിരിയുന്നു, ഓരോന്നും, root r -ന്റെ ഒരു child ആയ ഒരു tree ആണ്.

A tree has no cycles, because a cycle would give two different paths between two nodes. Choosing one node as the **root** turns it into the familiar **rooted tree**, drawn upside down with the root on top. The definition is recursive: a tree is a root plus a collection of smaller trees. A **general tree** lets any node have any number of children (e.g. a family tree). Trees are **non-linear** and **hierarchical**: there is no single first or last element, only a parent-child ordering from the root (top) to the leaves (bottom).

മലയാളത്തിൽ

ഒരു ഓഫീസിലെ hierarchy ആലോചിക്കൂ: MD മുകളിൽ (root), അദ്ദേഹത്തിന് കീഴിൽ managers, അവർക്ക് കീഴിൽ staff. താഴെ ആർക്കും കീഴുദ്യോഗസ്ഥർ ഇല്ലെങ്കിൽ അവർ leaf. Array പോലെ "ആദ്യത്തേത്, രണ്ടാമത്തേത്" എന്ന linear order ഇവിടെ ഇല്ല, parent-child ബന്ധം മാത്രം; അതുകൊണ്ടാണ് tree non-linear, hierarchical data structure ആകുന്നത്.



A rooted general tree. R is the root, D, E, B, F are leaves, A and C are internal nodes.

Term	Meaning (IGNOU)	In the figure
Root	node without a parent	R
Leaf	node with no children	D, E, B, F
Internal node	node with at least one child (non-leaf)	R, A, C
Edge	link between a parent and a child	6 edges
Level	root is at level 1, its children at level 2, and so on	D is at level 3
Depth of a node M	length (number of edges) of the path from the root to M	depth of D = 2
Depth (height) of a tree	number of levels in it	3
Degree of a node	number of children of the node	$\text{deg}(R) = 3, \text{deg}(A) = 2$
Branching factor	maximum number of children of any node (2 means binary tree)	3
Breadth of a level	number of nodes at that level	level 2 has 3
Siblings	nodes with the same parent	A, B, C

Level of a node = depth of the node + 1 (root: depth 0, level 1)

Number of edges = number of nodes - 1

EXAMPLE

For the tree in the figure, count nodes and edges and check the edge rule.

Nodes: R, A, B, C, D, E, F = 7. Edges: R-A, R-B, R-C, A-D, A-E, C-F = 6 = 7 - 1. ✓ Every node except the root has exactly one edge to its parent, so $e = n - 1$ for every tree.

Exam tip: IGNOU counts levels from 1 (root = level 1) but measures the depth of a node in edges (root depth = 0). State your convention in one line in the answer, and you will not lose marks.

Full trees, complete trees and counting properties

Full tree: a tree in which all the leaves are at the same level and all the non-leaf nodes have the same degree.

Complete tree: a k-ary position tree in which all levels are filled from left to right.

Full tree: എല്ലാ leaf-ഉം ഒരേ level-ൽ വരുന്നതും, leaf അല്ലാത്ത എല്ലാ node-കൾക്കും ഒരേ degree ഉള്ളതുമായ tree.

Complete tree: എല്ലാ level-ഉം ഇടത്തുനിന്ന് വലത്തോട്ട് ക്രമമായി നിറച്ച k-ary position tree.

In a full tree of degree d every internal node has exactly d children and the tree ends evenly, so the number of nodes grows like a geometric series: 1 at the first level, d at the second, d^2 at the third, and so on. Besides binary trees, IGNOU lists other specialised trees: binary search trees, AVL trees, red-black trees and 2-3 trees. A tree is also a **dynamic** data structure: implemented with pointers, it grows and shrinks while the program runs.

മലയാളത്തിൽ

ഒരു chain message ആലോചിക്കൂ: ഒരാൾ d പേർക്ക് അയക്കുന്നു, ഓരോരുത്തരും വീണ്ടും d പേർക്ക്. Level 1-ൽ 1 ആൾ, level 2-ൽ d , level 3-ൽ d^2 . ഇതാണ് full tree-ന്റെ count. ഇതിനൊപ്പം ഓർക്കേണ്ട ഏറ്റവും പ്രധാന rule: ഏത് tree-ലും edges = nodes - 1, കാരണം root ഒഴികെ ഓരോ node-നും parent-ലേക്ക് ഒരു edge മാത്രം.

Level h of a full tree has d^{h-1} nodes

First h levels: $1 + d + d^2 + \dots + d^{h-1} = (d^h - 1)/(d - 1)$ nodes

Edges = nodes - 1

Full m -ary tree (every internal node has exactly m children), i internal nodes, l leaves, n nodes:

$n = m \cdot i + 1$, $l = (m - 1) \cdot i + 1$, $i = (l - 1)/(m - 1)$, $l = ((m - 1)n + 1)/m$

EXAMPLE 1: NODE COUNT OF A FULL TREE

A full tree has degree $d = 3$ and height $h = 3$. How many nodes, leaves and edges?

Levels: 1, 3, 9 nodes. Total = $(3^3 - 1)/(3 - 1) = 26/2 = 13$. Leaves are the last level: $3^2 = 9$. Edges = $13 - 1 = 12$.

EXAMPLE 2 (CHECK YOUR PROGRESS 1, Q1-Q3)

(a) A tree has 45 edges. Vertices = $45 + 1 = 46$.

(b) A full 4-ary tree has 100 leaves. $i = (l - 1)/(m - 1) = 99/3 = 33$ internal vertices. Check: $n = 4 \cdot 33 + 1 = 133 = 33 + 100 \checkmark$.

(c) A full 3-ary tree has 100 internal vertices. $l = (m - 1)i + 1 = 2 \cdot 100 + 1 = 201$ leaves. Check: $n = 3 \cdot 100 + 1 = 301 = 100 + 201 \checkmark$.

EXAMPLE 3 (CHECK YOUR PROGRESS 1, Q4): PROOF

Show that a full m -ary tree with v vertices has $((m - 1)v + 1)/m$ leaves.

Every internal vertex has m children, and every vertex except the root is a child of exactly one internal vertex. So $v = m \cdot i + 1$, giving $i = (v - 1)/m$.

Leaves = all vertices - internal vertices: $l = v - (v - 1)/m = (mv - v + 1)/m = ((m - 1)v + 1)/m$. ■

Exam tip: The unit says "a tree of height h and degree d has at most $d^h - 1$ elements". That bound is reached only when $d = 2$; for larger d it is far too loose. The exact maximum is $(d^{h+1} - 1)/(d - 1)$; e.g. $d = 3, h = 2$ allows at most $1 + 3 + 9 = 13$ nodes, while $3^2 - 1 = 8$. Also note: the Check Your Progress formulas use "full m -ary" in the textbook sense (every internal node has exactly m children), which is weaker than the unit's "full tree" (all leaves on one level). The unit's answer key gives no answer for Q4; the proof above is the standard one.

Implementation of a general tree

Linked representation of a k -ary tree: each node stores one data field and k link (pointer) fields, one for each possible child; unused links are NULL.

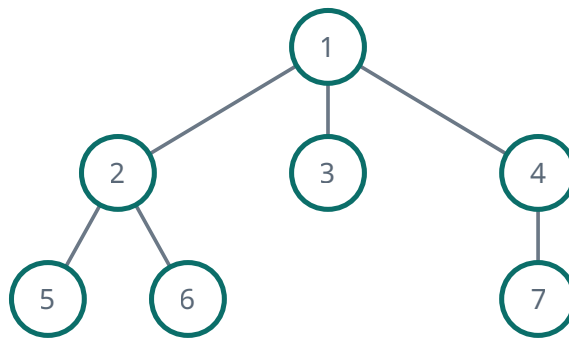
k -ary tree-ന്റെ linked representation: ഓരോ node-യും ഒരു data field-ഉം, ഓരോ child-നും ഒന്ന് വീതം k link (pointer) field-കളും; ഉപയോഗിക്കാത്ത link-കൾ NULL.

To insert into a general tree, first find the desired parent, then add the new node to that parent's child list. Nodes are usually inserted one at a time, but since every node is itself a tree, a whole sub-tree can also be built first and attached later. Because nodes are added and deleted dynamically, trees are normally implemented with linked lists. The number of pointers per node depends on the **maximum degree** of the tree (Figure 6.4: Data | link1 | link2 | ... | link k).

മലയാളത്തിൽ

ഒരു പുതിയ ജോലിക്കാരനെ ചേർക്കാൻ ആദ്യം അയാളുടെ boss ആരെന്ന് കണ്ടെത്തുന്നു, പിന്നെ boss-ന്റെ team list-ൽ ചേർക്കുന്നു; tree insertion-ഉം അങ്ങനെ തന്നെ. ഒരാൾക്ക് പരമാവധി 3 കീഴ്വ്യാഗസ്ഥർ എങ്കിൽ ഓരോ node-ലും 3 pointer വേണം (3-ary tree). ആരും ഇല്ലാത്ത സ്ഥാനത്ത് pointer NULL ആയിരിക്കും.

Data	link1	link2	...	link k
------	-------	-------	-----	--------



A 3-ary tree (like Figure 6.5): every node carries one data item and three child pointers.

EXAMPLE

Store the 3-ary tree above with nodes of the form [data | link1 | link2 | link3]. How many links are NULL?

Node 1: [1 | → 2 | → 3 | → 4]; node 2: [2 | → 5 | → 6 | NULL]; node 3: [3 | NULL | NULL | NULL]; node 4: [4 | → 7 | NULL | NULL]; nodes 5, 6, 7: all three links NULL.

Total links = $7 \times 3 = 21$. Used links = edges = $7 - 1 = 6$. NULL links = $21 - 6 = 15$. In general a k-ary linked tree with n nodes wastes $nk - (n - 1)$ links, which is why this scheme is costly for large k.

Tree traversals: preorder, postorder, inorder, level order

Traversal: visiting every node of a tree exactly once in a systematic order.

Preorder: visit the node, then its sub-trees. **Postorder:** visit the sub-trees, then the node. **Inorder:** left sub-tree, node, right sub-tree.

Traversal: ഒരു tree-യിലെ ഓരോ node-ഉം ഒരു നിശ്ചിത ക്രമത്തിൽ കൃത്യം ഒരു തവണ visit ചെയ്യൽ.

Preorder: ആദ്യം node, പിന്നെ sub-tree-കൾ. **Postorder:** ആദ്യം sub-tree-കൾ, അവസാനം node. **Inorder:** left sub-tree, node, right sub-tree.

The name tells you when the root is visited: **pre** = before the children, **post** = after them, **in** = in between.

Reading a book is preorder: you read a chapter heading, then all its sections, before the next chapter (Figure 6.6).

Finding the disk space used by a directory is postorder: you need the sizes of all files inside it first (Figure 6.7).

Each node is visited once, so all three take $T(n) = O(n)$. A fourth, less common order is **level order**: all nodes of level 1, then level 2, and so on, left to right.

മലയാളത്തിൽ

പുസ്തകം വായിക്കുമ്പോൾ ആദ്യം chapter title, പിന്നെ അതിലെ sections; ഇതാണ് preorder. ഒരു folder എത്ര MB എന്ന് അറിയാൻ ആദ്യം അതിലെ files-ന്റെ size കൂട്ടണം, അവസാനം folder; ഇതാണ് postorder. Inorder-ൽ ഇടതുഭാഗം, പിന്നെ node, പിന്നെ വലതുഭാഗം. Level order എന്നാൽ സ്കൂൾ assembly-യിലെ പോലെ ഓരോ വരിയും ഇടത്തുനിന്ന് വലത്തോട്ട്.

Preorder(T): visit root; preorder each sub-tree, left to right

Postorder(T): postorder each sub-tree, left to right; visit root

Inorder(T) (binary): inorder(left); visit root; inorder(right)

Level order: level 1, level 2, ..., each level left to right

Time for each: $O(n)$

EXAMPLE 1: READING A BOOK (FIGURE 6.6)

book has children Preface, Chapter 1, Chapter 8, Summary; Chapter 1 has Section 1 (with Section 1.1) and Section 4 (with Section 4.1, which has Section 4.1.1, and Section 4.2).

Preorder: book, Preface, Chapter 1, Section 1, Section 1.1, Section 4, Section 4.1, Section 4.1.1, Section 4.2, Chapter 8, Summary. This is exactly the reading order.

Postorder: Preface, Section 1.1, Section 1, Section 4.1.1, Section 4.1, Section 4.2, Section 4, Chapter 1, Chapter 8, Summary, book. This is the order in which you could add up page counts, parts before wholes.

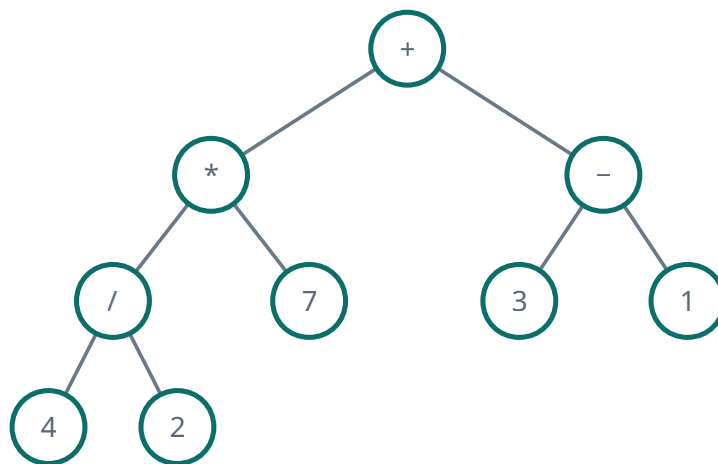


Figure 6.8: expression tree for $((4 / 2) * 7) + (3 - 1)$. Operators are internal nodes, operands are leaves.

EXAMPLE 2: THE EXPRESSION TREE OF FIGURE 6.8

Preorder (root, left, right): $+ * / 4 2 7 - 3 1$ (prefix form)

Postorder (left, right, root): $4 2 / 7 * 3 1 - +$ (postfix form)

Inorder (left, root, right): $4 / 2 * 7 + 3 - 1$; with brackets, $((4 / 2) * 7) + (3 - 1) = 14 + 2 = 16$.

Level order: $+ * - / 7 3 1 4 2$.

Exam tip: The unit prints the inorder of Figure 6.8 as "- (((4 / 2) * 7) + (3 - 1))". The leading "-" and the extra bracket are misprints; the correct inorder is $4 / 2 * 7 + 3 - 1$, i.e. $((4 / 2) * 7) + (3 - 1)$. Remember: preorder of an expression tree = prefix, inorder = infix, postorder = postfix.

Binary trees and their properties

Binary tree: a finite set of nodes that is either empty or consists of a root and two disjoint binary trees, called the left sub-tree and the right sub-tree.

Binary tree: ഒന്നുകിൽ empty ആയ, അല്ലെങ്കിൽ ഒരു root-ഉം left sub-tree, right sub-tree എന്ന രണ്ട് disjoint binary tree-കളും ചേർന്ന node-കളുടെ finite set.

So every node of a binary tree has **at most two** children, and they are distinguished as left and right. Formally the set is empty or partitioned into T_0 (the root), T_l and T_r (left and right binary trees). Binary trees model two-way decisions: yes/no, on/off, higher/lower.

മലയാളത്തിൽ

ഓരോ node-നും പരമാവധി രണ്ടു മക്കൾ: ഇടത്തേതും വലത്തേതും. "ചായയോ കാപ്പിയോ?", "പാസായോ തോറ്റോ?" പോലെ രണ്ട് വഴികളുള്ള തീരുമാനങ്ങൾ binary tree ആയി വരയ്ക്കാം. ഇടത്, വലത് എന്നത് ഇവിടെ പ്രധാനമാണ്; ഒരു child മാത്രമുണ്ടെങ്കിലും അത് left ആണോ right ആണോ എന്നു പറയണം.

n nodes $\Rightarrow$ exactly $n - 1$ edges

Height h (levels) $\Rightarrow$ at most $2^h - 1$ nodes

n nodes $\Rightarrow \lceil \log_2(n + 1) \rceil \leq \text{height} \leq n$

n nodes at level $l \Rightarrow$ at most 2^n nodes at level $l + 1$ (so level l has at most 2^{l-1} nodes)

Depth d (root depth 0) $\Rightarrow$ at most $2^0 + 2^1 + \dots + 2^d = 2^{d+1} - 1$ nodes

n nodes $\Rightarrow 2n$ links, of which $n + 1$ are NULL

EXAMPLE 1

What are the minimum and maximum heights of a binary tree with $n = 10$ nodes? With 7 nodes?

Max = $n = 10$ (every node has one child: a chain). Min = $\lceil \log_2(11) \rceil = \lceil 3.46 \rceil = 4$, since 3 levels hold at most 7 nodes.

For $n = 7$: max 7, min $\lceil \log_2 8 \rceil = 3$ (the full binary tree of height 3).

EXAMPLE 2 (CHECK YOUR PROGRESS 2, Q4)

Show that a binary tree with N nodes has $N + 1$ NULL links.

Each node has 2 link fields, so there are $2N$ links. A link is non-NULL exactly when it points to a child, i.e. it is an edge, and there are $N - 1$ edges. NULL links = $2N - (N - 1) = N + 1$.

Check with Figure 6.16 (next topics): $N = 9$, NULL links = C (2) + G, H, I, J (2 each) = $10 = 9 + 1 \checkmark$.

Exam tip: The unit words Q4 as "N + 1 children with both links null (leaf node)". That is wrong: Figure 6.16 has 9 nodes but only 5 leaves. What is true is that there are N + 1 NULL links. (For a tree where every node has 0 or 2 children, the leaf count is (N + 1)/2.) Also, the unit's "total number of nodes with depth d is $2^{d+1} - 1$ " is the maximum, reached only when every level is full.

Full and complete binary trees

Full binary tree: a binary tree of height h that has exactly $2^h - 1$ nodes (every level completely filled).

Complete binary tree: a binary tree of height d in which all levels except possibly level d are full, and the nodes of level d are as far left as possible.

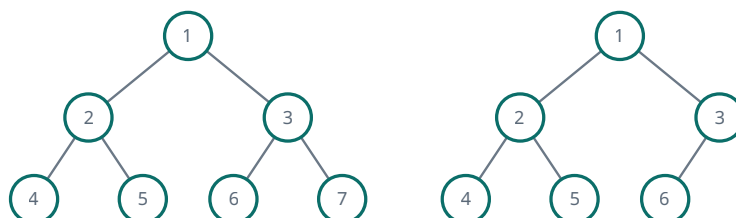
Full binary tree: height h ഉള്ളതും കൃത്യം $2^h - 1$ node-കൾ ഉള്ളതുമായ binary tree (എല്ലാ level-ഉം പൂർണ്ണമായി നിറഞ്ഞത്).

Complete binary tree: height d ഉള്ള binary tree-യിൽ അവസാന level d ഒഴികെ എല്ലാ level-ഉം നിറഞ്ഞതും, level d-യിലെ node-കൾ പരമാവധി ഇടത്തോട്ട് ചേർന്നിരിക്കുന്നതും.

A complete binary tree is what you get by filling nodes in **level order, left to right**, and stopping anywhere. Every full binary tree is complete, but not every complete tree is full. Complete trees matter because they can be stored in an array with no gaps (topic 9).

മലയാളത്തിൽ

ബസ്സിൽ സീറ്റ് നിറയുന്നതു പോലെ: മുൻനിര മുഴുവൻ നിറഞ്ഞ ശേഷമേ അടുത്ത നിരയിലേക്ക്, ആ നിരയിലും ഇടത്തുനിന്ന് തുടങ്ങി. അങ്ങനെ എവിടെ നിർത്തിയാലും complete tree. എല്ലാ നിരയും മുഴുവൻ നിറഞ്ഞാൽ അത് full tree.



Full binary tree, $h = 3$: $2^3 - 1 = 7$ nodes.

Complete (not full): last level filled from the left.

EXAMPLE

Classify: (a) the tree of Figure 6.12 (* with children + and 3; + with 4 and 5); (b) the tree of Figure 6.16.

(a) Levels 1 and 2 are full, level 3 has 4 and 5 at the leftmost positions. **Complete**, not full (5 nodes, not 7).

(b) Level 3 has D and E but C has no children, while level 4 has G, H, I, J. Level 3 is not full (positions under C are empty), so it is **neither complete nor full**.

Exam tip: Some books call IGNOU's "full binary tree" a perfect binary tree and use "full" for "every node has 0 or 2 children". In the IGNOU exam, use IGNOU's definition ($2^h - 1$ nodes).

Linked implementation and creation of a binary tree

Linked representation of a binary tree: each node is a structure holding a value and two pointers, leftchild and rightchild; a missing child is a NULL pointer.

Binary tree-ന്റെ linked representation: ഓരോ node-ഉം ഒരു value-ഉം leftchild, rightchild എന്ന രണ്ട് pointer-കളും ഉള്ള structure; ഇല്ലാത്ത child NULL pointer.

A node looks like [left child | value | right child] (Figure 6.10); a leaf has both pointers NULL. The unit builds the tree with a simple rule: compare the new element with the current node; if it is smaller or equal go left, otherwise go right; when the chosen side is empty, attach the new node there. The same logic is later used to **search** for a key. This rule actually builds a **binary search tree**.

മലയാളത്തിൽ

ലൈബ്രറിയിൽ പുസ്തകം വെക്കുന്നത് പോലെ: പുതിയ നമ്പർ നിലവിലെ node-നെക്കാൾ ചെറുതാണെങ്കിൽ ഇടത്തോട്ട്, വലുതാണെങ്കിൽ വലത്തോട്ട്. ഒഴിഞ്ഞ സ്ഥലം കിട്ടുന്നതുവരെ ഇങ്ങനെ താഴോട്ട് പോകുന്നു, അവിടെ പുതിയ node ഇരുത്തുന്നു. പിന്നീട് തിരയുമ്പോഴും ഇതേ വഴി തന്നെ.

Step 1: if new value $\leq$ current value, go to Step 2, else Step 3

Step 2: if current has no left sub-tree, make new its left child; else current = left child, go to Step 1

Step 3: if current has no right sub-tree, make new its right child; else current = right child, go to Step 1

EXAMPLE 1: TRACING THE INSERTION

Insert 14, 10, 16, 8, 11, 15, 18 in that order.

14 becomes the root. $10 \leq 14 \rightarrow$ left of 14. $16 > 14 \rightarrow$ right of 14. $8 \leq 14 \rightarrow$ go to 10; $8 \leq 10 \rightarrow$ left of 10. $11 \leq 14 \rightarrow$ go to 10; $11 > 10 \rightarrow$ right of 10. $15 > 14 \rightarrow$ go to 16; $15 \leq 16 \rightarrow$ left of 16. $18 > 14 \rightarrow$ 16; $18 > 16 \rightarrow$ right of 16.

The result is exactly the BST of Figure 6.14 (drawn in topic 13).

EXAMPLE 2: COMPLETE C PROGRAM (PROGRAM 6.1 + 6.2 TO 6.4, CORRECTED)

The program below creates the tree of Example 1 using the unit's `create_tree` and prints the three traversals.

```

#include <stdio.h>
#include <stdlib.h>

struct NODE
{
    struct NODE *left;
    int value;
    struct NODE *right;
};

struct NODE *make_node(int v)
{
    struct NODE *n = malloc(sizeof(struct NODE));
    if (n == NULL) {
        printf("Out of memory\n");
        exit(1);
    }
    n->value = v;
    n->left = n->right = NULL;
    return n;
}

void create_tree(struct NODE *curr, struct NODE *new)
{
    if (new->value <= curr->value) {
        if (curr->left != NULL)
            create_tree(curr->left, new);
        else
            curr->left = new;
    } else {
        if (curr->right != NULL)
            create_tree(curr->right, new);
        else
            curr->right = new;
    }
}

void inorder(struct NODE *curr)
{
    if (curr == NULL) return;
    inorder(curr->left);
    printf("%d ", curr->value);
    inorder(curr->right);
}

void preorder(struct NODE *curr)
{
    if (curr == NULL) return;
    printf("%d ", curr->value);
    preorder(curr->left);
    preorder(curr->right);
}

```

```

}

void postorder(struct NODE *curr)
{
    if (curr == NULL) return;
    postorder(curr->left);
    postorder(curr->right);
    printf("%d ", curr->value);
}

int main(void)
{
    int keys[] = {14, 10, 16, 8, 11, 15, 18};
    int n = sizeof(keys) / sizeof(keys[0]);
    int i;
    struct NODE *root = make_node(keys[0]);

    for (i = 1; i < n; i++)
        create_tree(root, make_node(keys[i]));

    printf("Inorder  : "); inorder(root);   printf("\n");
    printf("Preorder  : "); preorder(root);  printf("\n");
    printf("Postorder: "); postorder(root); printf("\n");
    return 0;
}

```

```

Inorder  : 8 10 11 14 15 16 18
Preorder  : 14 10 8 11 16 15 18
Postorder: 8 11 10 15 18 16 14

```

Exam tip: The unit's Program 6.1 declares `create_tree(...)` with no return type. Old C assumed `int`, but C99 and later reject this ("return type defaults to `int`" is an error in gcc), and the function returns nothing. Write `void create_tree(...)`, as above. Also note that it sends equal keys to the left (`<=`), while the BST definition in 6.8 assumes distinct keys.

Array representation of a binary tree

Array (sequential) representation: the nodes of a complete binary tree are numbered 0, 1, ..., $n - 1$ from top to bottom and left to right, and the item of node i is stored in $A[i]$.

Array (sequential) representation: complete binary tree-യിലെ node-കൾക്ക് മുകളിൽ നിന്ന് താഴോട്ടും ഇടത്തുനിന്ന് വലത്തോട്ടും 0, 1, ..., $n - 1$ എന്ന് നമ്പർ ഇട്ട്, node i -യുടെ item $A[i]$ -ൽ സൂക്ഷിക്കുന്നു.

With this numbering no pointers are needed: the parent and children of node i are found by arithmetic. For a tree that is not complete, the empty positions must be left blank in the array, which wastes space. The unit also shows a second table form (Figure 6.11) where each row stores the item and the indices of its left and right children, with -1 meaning "no child".

മലയാളത്തിൽ

ഒരു apartment-ലെ flat നമ്പറുകൾ പോലെ: flat i -യുടെ താഴെ $2i + 1$, $2i + 2$ എന്ന flats. അപ്പോൾ ആരുടെ "മക്കൾ" എവിടെ എന്നറിയാൻ pointer വേണ്ട, കണക്ക് മതി. Tree complete അല്ലെങ്കിൽ ഇടയ്ക്ക് ഒഴിഞ്ഞ flats (ഒഴിഞ്ഞ array cells) വരും.

Parent of i : $\lfloor (i - 1)/2 \rfloor$ (for $i > 0$)

Left child of i : $2i + 1$

Right child of i : $2i + 2$

(an index $\geq n$, or an empty cell, means "no such node")

EXAMPLE 1 (CHECK YOUR PROGRESS 2, Q3): FIGURE 6.12

The tree $^*(+ (4, 5), 3)$ is complete, so level order gives the array directly.

Index	0	1	2	3	4
Item	*	+	3	4	5
Left child ($2i + 1$)	1	3	-1	-1	-1
Right child ($2i + 2$)	2	4	-1	-1	-1

Check: parent of 4 (the item 5) = $\lfloor (4 - 1)/2 \rfloor = 1$, which is +. ✓

EXAMPLE 2: FIGURE 6.16, TWO WAYS

(a) Index-table form (the unit's Figure 6.11): number the nodes 0–8 in level order and store child indices.

Node #	0	1	2	3	4	5	6	7	8
Item	A	B	C	D	E	G	H	I	J
Left	1	3	-1	5	7	-1	-1	-1	-1
Right	2	4	-1	6	8	-1	-1	-1	-1

(b) Formula form ($2i + 1$, $2i + 2$): C's children would be at 5 and 6, which stay empty, so D's children go to 7, 8 and E's to 9, 10.

Index	0	1	2	3	4	5	6	7	8	9	10
A[i]	A	B	C	D	E	-	-	G	H	I	J

11 cells for 9 nodes: the two blanks are the cost of a non-complete tree.

Exam tip: The unit's Figure 6.11 numbers the nodes of Figure 6.16 consecutively (D's children at 5, 6), so its left/right columns do not follow the $2i + 1$ / $2i + 2$ formula stated just above it; that formula holds only for a complete tree. Write the parent formula with the floor: $\lfloor (i - 1)/2 \rfloor$ (integer division in C).

Binary tree traversals: recursive implementation

Inorder: traverse the left sub-tree, visit the root, traverse the right sub-tree. **Preorder:** root, left, right.

Postorder: left, right, root.

Inorder: left sub-tree traverse ചെയ്യുക, root visit ചെയ്യുക, right sub-tree traverse ചെയ്യുക. **Preorder:** root, left, right. **Postorder:** left, right, root.

All three follow the same recursive pattern; the only difference is when the root is processed. Because a binary tree is defined recursively, the traversal code is naturally recursive and very short. Each call handles one node, so the time is $O(n)$; the recursion stack uses space proportional to the height.

മലയാളത്തിൽ

മൂന്നിലും ഇടത്തേതിനെ വലത്തേതിനു മുമ്പ് നോക്കും; വ്യത്യസ്തം root-നെ എപ്പോൾ print ചെയ്യുന്നു എന്നതിൽ മാത്രം: മുമ്പ് (pre), നടുവിൽ (in), ശേഷം (post). ഓർക്കാൻ എളുപ്പം: "root എവിടെ" എന്ന് പേര് തന്നെ പറയുന്നു.

Inorder: L N R Preorder: N L R Postorder: L R N (N = visit node)

EXAMPLE 1: FIGURE 6.12, INORDER STEP BY STEP

Start at $*$; it has a left sub-tree, go to $+$; $+$ has a left sub-tree, go to 4. 4 has no left child: visit 4; no right child, go back. Visit $+$; go to its right child 5: visit 5. Back to $*$: visit $*$; go right to 3: visit 3.

Inorder: $4 + 5 * 3$. Preorder: $* + 4 5 3$. Postorder: $4 5 + 3 *$.

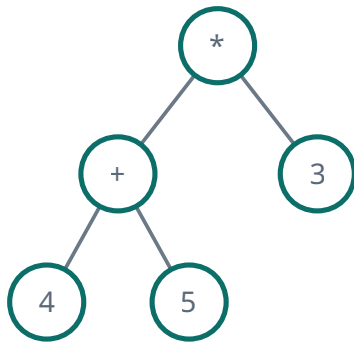


Figure 6.12

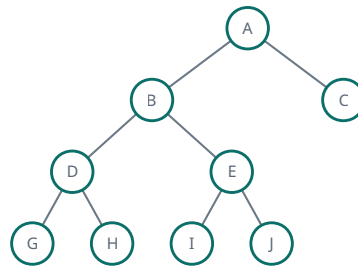


Figure 6.16

EXAMPLE 2 (CHECK YOUR PROGRESS 2, Q1 AND Q2): FIGURE 6.16

(a) Leaves: C, G, H, I, J. (b) Sibling of J: I. (c) Parent of G: D. (d) Depth (height) of the tree: 4 levels. (e) Level of J: 4.

Preorder: **A B D G H E I J C**

Inorder: **G D H B I E J A C**

Postorder: **G H D I J E B C A**

Level order: **A B C D E G H I J**

Exam tip: The unit's answer key has three errors for Figure 6.16. (1) Leaves are C, G, H, I, J; the key forgets C (it has no children). (2) It gives preorder "ABDCHEIJC"; the correct preorder is A B D G H E I J C. (3) It gives inorder "GDHBI EFAC", which contains an F that is not in the tree and misses J; the correct inorder is G D H B I E J A C. Its postorder and level order are right. Also, Programs 6.2–6.4 have no return type (write void), print values with no space between them, and crash if called on an empty tree; the version in topic 8 tests `curr == NULL` first.

Non-recursive traversal using a stack

Non-recursive (iterative) traversal: a traversal done with a loop and an explicit stack of node pointers, instead of recursive calls.

Non-recursive (iterative) traversal: recursive call-കൾക്ക് പകരം ഒരു loop-ഉം node pointer-കളുടെ ഒരു explicit stack-ഉം ഉപയോഗിച്ചുള്ള traversal.

Recursion secretly uses the system stack; the iterative version keeps that stack itself. For **preorder**: push the root; repeatedly pop a node, visit it, push its right child and then its left child. The left child is pushed last so it is popped first. In the worst case the stack grows to about $n/2$ entries.

മലയാളത്തിൽ

ഒരു വീട്ടിൽ പല മുറികൾ പരിശോധിക്കുമ്പോൾ "പിന്നെ നോക്കാം" എന്ന മുറികൾ ഒരു ലിസ്റ്റിൽ മുകളിലേക്ക് എഴുതിവെക്കുന്നു; അവസാനം എഴുതിയത് ആദ്യം നോക്കുന്നു (LIFO). Right child-നെ ആദ്യം push ചെയ്ത്, left child-നെ പിന്നീട് push ചെയ്യുന്നതുകൊണ്ട് left ആദ്യം pop ആകും; അങ്ങനെ root, left, right എന്ന preorder കിട്ടുന്നു.

Preorder: push root; while stack not empty { v = pop; if v ≠ NULL { visit v; push v → right; push v → left } }
Inorder: curr = root; while curr ≠ NULL or stack not empty { while curr ≠ NULL { push curr; curr = curr → left }; curr = pop; visit curr; curr = curr → right }

EXAMPLE 1: STACK TRACE OF PREORDER ON FIGURE 6.16

Pop / visit	Push (right , then left)	Stack after (bottom → top)
(start)	A	A
A	C, B	C B
B	E, D	C E D
D	H, G	C E H G
G	–	C E H
H	–	C E
E	J, I	C J I
I	–	C J
J	–	C
C	–	empty

Visit order: A B D G H E I J C, same as the recursive preorder. Largest stack: 4 entries.

EXAMPLE 2: C PROGRAM (PROGRAM 6.5, CORRECTED, PLUS INORDER)

```

#include <stdio.h>
#include <stdlib.h>

#define MAX 100

struct NODE
{
    struct NODE *left;
    char value;
    struct NODE *right;
};

struct NODE *make_node(char v, struct NODE *l, struct NODE *r)
{
    struct NODE *n = malloc(sizeof(struct NODE));
    if (n == NULL) {
        printf("Out of memory\n");
        exit(1);
    }
    n->value = v;
    n->left = l;
    n->right = r;
    return n;
}

void preorder_iter(struct NODE *root)
{
    struct NODE *stack[MAX];
    int top = -1;
    struct NODE *v;

    if (root == NULL) return;
    stack[++top] = root;
    while (top >= 0) {
        v = stack[top--];          /* pop */
        printf("%c ", v->value);   /* visit */
        if (v->right != NULL) stack[++top] = v->right;
        if (v->left != NULL) stack[++top] = v->left;
    }
    printf("\n");
}

void inorder_iter(struct NODE *root)
{
    struct NODE *stack[MAX];
    int top = -1;
    struct NODE *curr = root;

    while (curr != NULL || top >= 0) {
        while (curr != NULL) {    /* go as far left as possible */
            stack[++top] = curr;

```

```

        curr = curr->left;
    }
    curr = stack[top--];          /* pop */
    printf("%c ", curr->value);   /* visit */
    curr = curr->right;           /* then its right sub-tree */
}
printf("\n");
}

int main(void)
{
    /* Figure 6.16 */
    struct NODE *D = make_node('D', make_node('G', NULL, NULL), make_node('H', NULL, NULL));
    struct NODE *E = make_node('E', make_node('I', NULL, NULL), make_node('J', NULL, NULL));
    struct NODE *B = make_node('B', D, E);
    struct NODE *A = make_node('A', B, make_node('C', NULL, NULL));

    printf("Preorder (with a stack): ");
    preorder_iter(A);
    printf("Inorder (with a stack): ");
    inorder_iter(A);
    return 0;
}

```

```

Preorder (with a stack): A B D G H E I J C
Inorder (with a stack): G D H B I E J A C

```

Exam tip: The unit's algorithm checks "if v is not NULL" but its Program 6.5 does not: it pushes NULL children and then calls `visit(tree)` and `tree->right` on a NULL pointer, which crashes at the first leaf. Either check after popping (as in the algorithm) or push only non-NULL children (as in the program above). Program 6.5 also uses an undefined `stack_type` library; the version above uses a plain array stack so that it compiles on its own.

Threaded binary trees

Threaded binary tree: a binary tree in which every node that has no right child has a link, called a thread, to its inorder successor.

Threaded binary tree: right child ഇല്ലാത്ത ഓരോ node-ഉം thread എന്ന ഒരു link വഴി അതിന്റെ inorder successor-ലേക്ക് ചൂണ്ടുന്ന binary tree.

After finishing a node with no right sub-tree, an inorder traversal must "climb back up" to the next node; normally a stack or recursion remembers where to go. A thread stores that next node directly, so inorder traversal

needs **neither recursion nor a stack**, saving memory and time. The unit's node adds a third pointer for the thread.

മലയാളത്തിൽ

ഒരു treasure hunt-ൽ ഓരോ സ്ഥലത്തും "അടുത്തത് എവിടെ" എന്ന കുറിപ്പ് വെച്ചാൽ, പോയ വഴി ഓർത്തുവെക്കേണ്ട ആവശ്യമില്ല. Thread അതുപോലെയാണ്: right child ഇല്ലാത്ത node inorder-ൽ അടുത്ത node ഏതാണെന്ന് നേരിട്ട് കാണിക്കുന്നു. അതുകൊണ്ട് stack ഇല്ലാതെ തന്നെ traverse ചെയ്യാം.

```
struct NODE
{
    struct NODE *leftchild;
    int node_value;
    struct NODE *rightchild;
    struct NODE *thread; /* third pointer to its inorder successor */
};
```

EXAMPLE

Find the threads in Figure 6.16. Inorder: G D H B I E J A C.

Nodes with no right child: G, H, I, J, C. Their inorder successors: $G \rightarrow D$, $H \rightarrow B$, $I \rightarrow E$, $J \rightarrow A$, $C \rightarrow \text{none}$ (C is last, so its thread is NULL).

Traversal: start at the leftmost node G; follow the thread to D; D has a right child H, go to the leftmost node of that sub-tree (H); thread $H \rightarrow B$; B's right sub-tree starts at its leftmost node I; thread $I \rightarrow E$; E's right child J; thread $J \rightarrow A$; A's right child C; C's thread is NULL, stop. Order: G D H B I E J A C $\checkmark$.

Exam tip: The unit says the stack-free method "requires pointers to the parent node (called threaded binary tree)". That is loose: threads point to the inorder successor, not to the parent (for H the thread goes to B, its grandparent). Standard textbooks also avoid a third pointer: they reuse the empty right (and left) links as threads and keep a flag bit to tell a thread from a real child. The unit's struct is also missing the final semicolon after `};` it is added above.

Applications: BST, expression trees, game trees

Binary search tree (BST): a binary tree in which the key of every node is greater than all keys in its left sub-tree and smaller than all keys in its right sub-tree.

Expression tree: a binary tree whose leaves are operands and whose internal nodes are operators applied to their two sub-trees.

Binary search tree (BST): ഓരോ node-ന്റെയും key അതിന്റെ left sub-tree-യിലെ എല്ലാ key-കളെക്കാളും വലുതും right sub-tree-യിലെ എല്ലാ key-കളെക്കാളും ചെറുതും ആയ binary tree.

Expression tree: leaf-കൾ operand-കളും, internal node-കൾ രണ്ട് sub-tree-കളിൽ പ്രയോഗിക്കുന്ന operator-കളും ആയ binary tree.

Trees are everywhere: faster database search (BST, 2-3, AVL, red-black trees), games (minimax, decision and pathfinding trees), 3D graphics (quadtrees, octrees), arithmetic precedence trees in scripting languages, data compression (Huffman trees) and file systems (B-trees, sparse indexed trees, tries). A **game tree** such as tic-tac-toe (Figure 6.13) lists the positions reachable from each move; its problem is size: two choices per level in a 10-level game already need $2^{10} - 1 = 1023$ nodes. An inorder traversal of a **BST** prints the keys in sorted order. An **expression tree** is evaluated bottom-up: a leaf gives its value; an internal node evaluates its children and combines them with its operator (a postorder process).

മലയാളത്തിൽ

Phone-ലെ contact list-ൽ ഒരു പേര് തിരയുമ്പോൾ പകുതി ഒഴിവാക്കി മുന്നോട്ട് പോകുന്നതുപോലെയാണ് BST: ചെറുത് ഇടത്, വലുത് വലത്. Inorder-ൽ വായിച്ചാൽ keys sorted order-ൽ കിട്ടും. Expression tree-ൽ ആദ്യം താഴത്തെ ചെറിയ കണക്കുകൾ ചെയ്ത് മുകളിലേക്ക് ഉത്തരം കൊണ്ടുപോകുന്നു, ഇത് postorder തന്നെ. Chess, tic-tac-toe പോലുള്ള കളികളിൽ computer അടുത്ത നീക്കം തീരുമാനിക്കാൻ game tree ഉപയോഗിക്കുന്നു.

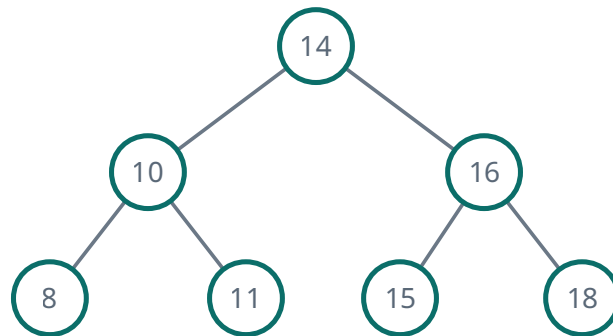


Figure 6.14: a binary search tree. Inorder: 8 10 11 14 15 16 18 (sorted).

EXAMPLE 1: SEARCHING THE BST OF FIGURE 6.14

Search for 15: $15 > 14 \rightarrow$ go right to 16; $15 < 16 \rightarrow$ go left to 15; found in 3 comparisons. Search for 12: $12 < 14 \rightarrow 10$; $12 > 10 \rightarrow 11$; $12 > 11 \rightarrow$ right of 11 is NULL, so 12 is **not present**.

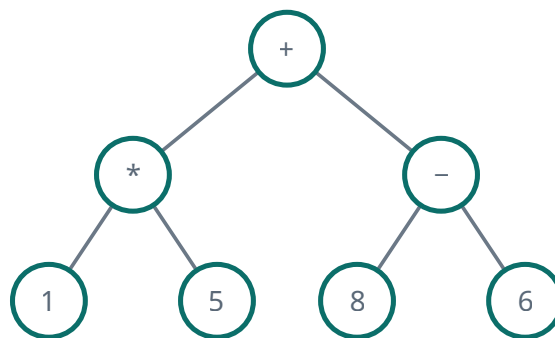


Figure 6.15: expression tree for $(1 * 5) + (8 - 6)$.

EXAMPLE 2: EVALUATING EXPRESSION TREES

Figure 6.15: * node $\rightarrow 1 \times 5 = 5$; - node $\rightarrow 8 - 6 = 2$; + root $\rightarrow 5 + 2 = 7$. Figure 6.8: $4 / 2 = 2$; $2 * 7 = 14$; $3 - 1 = 2$; $14 + 2 = 16$.

The same rule as a C program (the show function is an inorder traversal that adds brackets):

```

#include <stdio.h>
#include <stdlib.h>

struct NODE
{
    struct NODE *left;
    char op;          /* operator, or 0 for a leaf */
    int value;        /* used only by leaves */
    struct NODE *right;
};

struct NODE *leaf(int v)
{
    struct NODE *n = malloc(sizeof(struct NODE));
    if (n == NULL) exit(1);
    n->op = 0; n->value = v; n->left = n->right = NULL;
    return n;
}

struct NODE *opnode(char op, struct NODE *l, struct NODE *r)
{
    struct NODE *n = malloc(sizeof(struct NODE));
    if (n == NULL) exit(1);
    n->op = op; n->value = 0; n->left = l; n->right = r;
    return n;
}

int eval(struct NODE *t)
{
    int a, b;
    if (t->op == 0)                /* leaf: its value */
        return t->value;
    a = eval(t->left);              /* evaluate the children ... */
    b = eval(t->right);
    switch (t->op) {                /* ... then combine them */
        case '+': return a + b;
        case '-': return a - b;
        case '*': return a * b;
        case '/': return a / b;
    }
    return 0;
}

void show(struct NODE *t)          /* inorder with brackets */
{
    if (t->op == 0) { printf("%d", t->value); return; }
    printf("(");
    show(t->left);
    printf(" %c ", t->op);
    show(t->right);
    printf(")");
}

```

```

}

int main(void)
{
    /* Figure 6.15 and Figure 6.8 */
    struct NODE *f15 = opnode('+', opnode('*', leaf(1), leaf(5)),
                                opnode('-', leaf(8), leaf(6)));
    struct NODE *f8  = opnode('+', opnode('*', opnode('/', leaf(4), leaf(2)), leaf(7)),
                                opnode('-', leaf(3), leaf(1)));

    show(f15); printf(" = %d\n", eval(f15));
    show(f8);  printf(" = %d\n", eval(f8));
    return 0;
}

```

```

((1 * 5) + (8 - 6)) = 7
(((4 / 2) * 7) + (3 - 1)) = 16

```

Exam tip: "Inorder traversal of a BST gives the keys in ascending order" is a favourite 2-mark question; prove it with an example like Figure 6.14. The unit's pseudocode `inorder(root)` there also needs a base case: return when root is NULL.

Quick recap

- A tree is a connected acyclic graph; a rooted tree has one root and every other node has exactly one parent.
- Root: no parent. Leaf: no children. Level: root = 1. Height of a tree = number of levels. Depth of a node = edges from the root.
- Every tree with n nodes has $n - 1$ edges.
- Full tree of degree d and height h : level h has d^{h-1} nodes; total $(d^h - 1)/(d - 1)$.
- Full m -ary tree: $n = m^i + 1$, $l = (m - 1)i + 1$.
- General (k -ary) trees are stored as nodes with one data field and k child links.
- Preorder = N L R, inorder = L N R, postorder = L R N, level order = level by level; each is $O(n)$.
- Binary tree: each node has at most two children, left and right; height $h \Rightarrow$ at most $2^h - 1$ nodes; n nodes $\Rightarrow$ height between $\lceil \log_2(n + 1) \rceil$ and n .
- A binary tree with N nodes has $N + 1$ NULL links.
- Full binary tree: $2^h - 1$ nodes. Complete binary tree: filled level by level, left to right.
- Array form: children of i at $2i + 1$ and $2i + 2$, parent at $\lfloor (i - 1)/2 \rfloor$.
- Non-recursive preorder: pop, visit, push right then left. Threaded trees point each missing right link to the inorder successor, so no stack is needed.
- BST: left < node < right; its inorder traversal is sorted.
- Expression tree: preorder = prefix, inorder = infix, postorder = postfix; evaluate bottom-up.

Check yourself

Q. 1. If a tree has 45 edges, how many vertices does it have?

$$e = n - 1, \text{ so } n = 45 + 1 = \mathbf{46}.$$

Q. 2. A full 4-ary tree has 100 leaves. How many internal vertices does it have?

$$l = (m - 1)i + 1 \Rightarrow 100 = 3i + 1 \Rightarrow i = \mathbf{33}.$$

Q. 3. A full 3-ary tree has 100 internal vertices. How many leaves does it have?

$$l = (3 - 1) \cdot 100 + 1 = \mathbf{201}.$$

Q. 4. Prove that a full m -ary tree with v vertices has $((m - 1)v + 1)/m$ leaves.

Each of the i internal vertices has m children, and every vertex except the root is somebody's child, so $v = mi + 1$ and $i = (v - 1)/m$. Then $l = v - i = v - (v - 1)/m = ((m - 1)v + 1)/m$.

Q. 5. For Figure 6.16 find the leaves, the sibling of J, the parent of G, the depth of the tree and the level of J.

Leaves: C, G, H, I, J. Sibling of J: I. Parent of G: D. Depth: 4. Level of J: 4.

Q. 6. Give the preorder, inorder, postorder and level order traversals of Figure 6.16.

Preorder: A B D G H E I J C. Inorder: G D H B I E J A C. Postorder: G H D I J E B C A. Level order: A B C D E G H I J.

Q. 7. Give the array representation of the binary tree of Figure 6.12.

$A[0] = *$, $A[1] = +$, $A[2] = 3$, $A[3] = 4$, $A[4] = 5$. Left/right children: $0 \rightarrow 1, 2$; $1 \rightarrow 3, 4$; nodes 2, 3, 4 have -1, -1 (no children).

Q. 8. Show that a binary tree with N nodes has $N + 1$ NULL links.

There are $2N$ link fields. $N - 1$ of them are edges (every node except the root has one link pointing to it).
NULL links = $2N - (N - 1) = N + 1$.

Q. 9. Write the prefix, infix and postfix forms of the expression tree in Figure 6.15 and evaluate it.

Preorder (prefix): $+ * 1 5 - 8 6$. Inorder (infix): $1 * 5 + 8 - 6$. Postorder (postfix): $1 5 * 8 6 - +$. Value: $5 + 2 = 7$.

Q. 10. Insert 50, 30, 70, 20, 40, 60, 80 into an empty BST using the unit's creation rule. Give the preorder, inorder and postorder traversals.

Tree: 50 at the root; 30 (children 20, 40) on the left; 70 (children 60, 80) on the right.

Preorder: 50 30 20 40 70 60 80. Inorder: 20 30 40 50 60 70 80 (sorted, as expected for a BST). Postorder: 20 40 30 60 80 70 50.

Based on the IGNOU unit texts. Check definitions against the PDFs before the exam.